Reverse Neighbor Sliding and Order Selection for Efficient Multi-Proximity Graph Merging

Liuchang Jing
HKUST (GZ)
Guangzhou, Guangdong, China
ljing248@connect.hkust-gz.edu.cn

Mingyu Yang
HKUST (GZ) & HKUST
Guangzhou, Guangdong, China
myang250@connect.hkust-gz.edu.cn

Lei Li
HKUST (GZ) & HKUST
Guangzhou, Guangdong, China
thorli@ust.hk

Jianbin Qin
Shenzhen University
Shenzhen, Guangdong, China
qinjianbin@szu.edu.cn

Wei Wang
HKUST(GZ) & HKUST
Guangzhou, Guangdong, China
weiwcs@ust.hk

Abstract

Approximate k Nearest Neighbor (AKNN) search in high-dimensional space is a foundational problem in vector databases with widespread applications. Among the numerous AKNN indexes, Proximity Graph-based indexes achieve state-of-the-art search efficiency across various benchmarks. In many real-world scenarios, datasets are maintained as multiple segment-level graph indexes to support continuous writes and segment management. However, these fragmented indexes complicate maintenance and degrade search efficiency, making fast graph index merging essential. In this paper, we focus on the efficient merging of multiple existing graph indexes into a single one. To achieve this, we propose a Reverse Neighbor Sliding Merge (RNSM) that exploits structural information to boost merging efficiency. We further propose Merge Order Selection (MOS) to minimize total merge cost across multiple indexes by eliminating redundant operations. Experiments show that our approach yields up to a 3.86× speedup over existing index merge methods and a 9.92× speedup over index reconstruction, while maintaining comparable search performance. Moreover, our method scales to merging up to 50 sub-indexes on datasets of 100 million vectors, maintaining consistent speedups.

CCS Concepts

• **Information systems** → **Information retrieval**.

Keywords

Approximate Nearest Neighbor Search, Graph Index, Vector Databases, Information Retrieval

ACM Reference Format:
Liuchang Jing, Mingyu Yang, Lei Li, Jianbin Qin, and Wei Wang. 2027. Reverse Neighbor Sliding and Order Selection for Efficient Multi-Proximity Graph Merging. In *Proceedings of the 2027 ACM SIGMOD/PODS International Conference on Management of Data (SIGMOD '27).* ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '27, Huntington Beach, CA, USA
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2027/06
https://doi.org/XXXXXXX.XXXXXXX

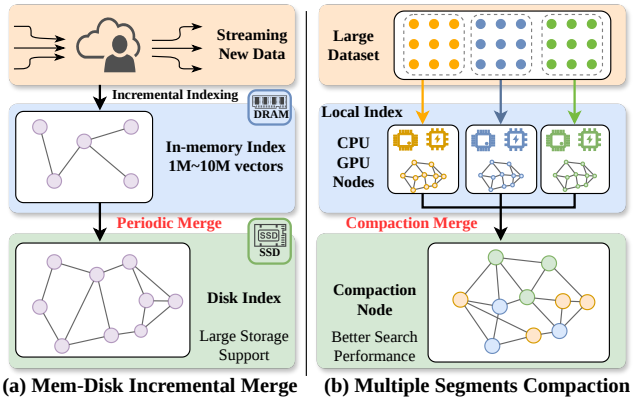
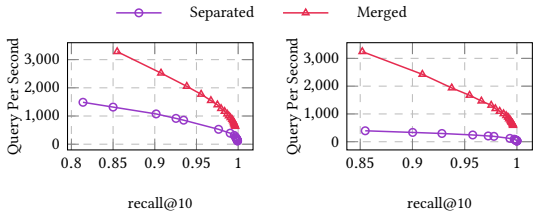


Figure 1: Examples of Multiple Indexes Merging Scenarios

1 Introduction

The k nearest neighbor (KNN) search in high-dimensional space is a fundamental problem that has received widespread attention due to its extensive applications in data mining [10], information retrieval [38, 46], recommendation systems [11, 48], and the Retrieval-Augmented Generation (RAG) of Large Language Models (LLMs) [35]. However, the cost of exact KNN search is extremely high for large-scale datasets [28]. Therefore, extensive research has focused on studying approximate k nearest neighbor (AKNN) search [2–4, 6, 8, 9, 12, 13, 15–24, 26, 31, 36, 39, 40, 43, 45, 47, 49, 53–56, 65, 69, 70], trading accuracy for search efficiency. Among various AKNN methods, graph-based indexes are the state-of-the-art solution, demonstrating comparable performance compared to other approaches [5, 37, 42, 62]. Graph-based methods represent each vector in the dataset as a node in a graph, where edges connecting nodes reflect proximity relationships designed to facilitate efficient navigation during search. Most graph-based methods construct a Proximity Graph (PG) through one of two approaches: (1) refining a random or approximate k nearest neighbor graph (KNNG) [16, 17, 45, 72], or (2) incrementally inserting nodes into an empty graph [39, 40].

However, both indexing methods incur substantial time costs for graph construction [5, 62] due to extensive distance computations.



(a) 10 Partitions of DEEP1M **(b) 50 Partitions of DEEP1M**

Figure 2: Separated and Merged index single-thread search performance. Sub-indexes are constructed on 10 and 50 equal-sized partitions of the DEEP1M dataset. Separated indicates searching sequentially on all partitions. Merged indicates searching a single index merged by our proposed method.

Therefore, commercial vector databases like Milvus [59] and BlendHouse [44] typically partition data *by design* to support dynamic and independent segment data management. Despite the scalability of this approach, directly searching multiple sub-indexes incurs a significant search performance degradation. This phenomenon is intuitive because searching m separate graph indexes independently requires traversing each from its entry point. Given that graph search complexity is logarithmic with respect to dataset size [17, 36, 45], querying m partitions scales the computational cost up to a factor of approximately m in the worst case. As shown in the experimental results in Fig 2, the search efficiency of the partitioned index with 10 partitions is only half that of the complete index, and this efficiency gap widens to nearly 1/6 with 50 partitions. Therefore, merging multiple indexes is a critical operation in various scenarios.

Fig 1 shows two index-merging scenarios. In *incremental index merging* of Fig 1(a), streaming data are indexed in a small index and periodically merged into the larger index of prior data. To support continuous writes within memory limits, commercial systems keep the large index on SSD and new segments (typically fewer than 10M vectors) in memory under an LSM architecture [44]. Existing methods treat this merge as costly incremental indexing [51, 81], which is computationally expensive. In *multiple segment compaction* of Fig 1(b), CPU or GPU nodes independently build and search local segment indexes [73, 77]. Periodic compaction reduces storage and management overhead and improves search, but existing systems rebuild the full index from scratch [50, 77], which introduces high computational cost. In both scenarios, speeding up index merging is crucial for efficiency.

Previous Methods. We categorize existing index merge methods into two families based on their strategies for connecting multiple sub-graph indexes: (1) **Overlap**-based [53, 68, 74] methods intentionally duplicate data points across multiple segments during sub-index construction. The sub-indexes can be merged by concatenating the neighbor lists of the overlapping nodes. (2) **Search**-based [40] treats the data in one index as queries to search in another index. The neighbors of the subgraph index node are updated with the search results. However, for already-built indexes, overlap-based methods require building indexes with duplicated vectors. Their merge cost also scales with the overlapping ratio, which is at least 2 [53]. Therefore, we focus on search-based methods that can also handle overlapping partitions: applying the neighbor pruning rule of the PG index to the concatenation of search results and existing neighbors from different indexes. Recent search-based methods [33, 66, 67, 78] focus on reducing the time cost of merging multiple existing graph indexes in main memory by reducing the Number of Distance Computations (NDC). This is a practical setting with growing DRAM capacity [61, 62] and distance computation dominating the cost.

State-of-the-art. A recent search-based study, FGIM [66], has emerged as the most efficient method for index merging. It consists of a 3-step pipeline: 1) determine which pair of partitions to merge, 2) accelerate the pairwise merge operation, and 3) refine the final merged index. For step 1, FGIM searches each vector in all other partitions for nearest neighbors, noted as **cross-querying**. In step 2, its **Minimum Candidate Set Strategy** assumes that the cross-querying **search effort** ef can be evenly allocated among $m-1$ other partitions for a good enough merge quality. Thus, each search is $1/(m-1)$ times cheaper. For step 3, with AKNN results of the previous step, indexes are roughly merged into a KNNG, noted as **KNNG transformation**. To recover the missing k nearest neighbors due to the cheaper cross-querying in step 2, the KNNG is further refined with NNDecent [14]. The refined KNNG is further pruned into a PG index. However, FGIM has issues in all three steps: (1) In cross-querying, the m^2 all-pairs merges contain redundant operations, since not all partition pairs need to be merged; (2) The assumption in Minimum Candidate Set Strategy limits its applicability to unbalanced partition sizes, where few sub-indexes could contribute significantly more neighbors than others therefore requiring larger ef; (3) KNNG refining takes $O(k^2 \cdot n)$ time [66], where n is the data size and k increases with m, limitating its scalability.

Our Solution. In this paper, we propose a novel search-based multi-index merging framework through two key techniques: *Reverse Neighbor Sliding Merge* (RNSM) and *Merge Order Selection* (MOS). We introduce them with 3 similar steps. In step 1, we propose MOS to reduce the m^2 merge operations. MOS treats partitions as vertices and pairwise merges as edges, and constructs a sparse and proximity-aware merge order graph that prioritizes the merge of closer partition pairs while preserving connectivity between partitions. For step 2, RNSM merges two partitions efficiently by reusing search results. Only selected vectors perform standard searches, their nearby vectors reuse the corresponding results as effective entry points for lower-cost searches. Thus, RNSM accelerates pairwise merging without sacrificing search quality or relying on uniform search-effort allocation of FGIM. For step 3, we execute the RNSM operations selected by MOS and directly update the merged PG, avoiding expensive KNNG refining and cross-querying with faster pairwise merge and fewer merge operations. Moreover, we further determine the direction of search-based merging by defining **source and target indexes**: a source index supplies the vectors that issue cross-index queries on the target index, *i.e.*, source indexes are merged into target indexes. We always merge smaller indexes into larger ones [60] and merge closer partition pairs first [1] in step 3.

Contribution. We summarize our main contributions as follows:

Problem Analysis and Motivation (§ 3). We explored the key factors in index merging and found that nearest neighbor connections are

Table 1: A Summary of Notations

Notation	Description
X_i	A set of points/vectors
x, y	A data point/vector
$\mathbb{R}^D$	D-dimensional Euclidean space
$\delta(x, y)$	Euclidean/L_2 distance between x and y
$G_i = (V_i, E_i)$	The graph index for X_i with vertex set V_i and edge set E_i
$\mathcal{L}(p, q)$	The local sliding NDC cost from point p to q
$N(v, G)$	The neighbors of v in G
$N_k(x, X)$	The k-nearest neighbors of x in X
$R_k(x, X)$	The reverse k-nearest neighbors of x in X

crucial, thus transforming the index merging problem into a search optimization problem. We also analyzed the patterns of multi-index merging, providing new insights into the merge order selection.

Efficient Two-Index Merge Method (§ 4). We propose RNSM, an efficient two-index merge method that leverages structural information from both the source and target indexes to accelerate merging. RNSM first identifies *pivots*: source-index vectors that perform standard searches in the target index and whose results are reused by nearby *followers* as entry points for local sliding.

Merge Order Selection (§ 5). We propose MOS, which formulates merge order selection as a graph optimization problem that minimizes merge cost with two constraints: preserving efficiency through degree control and search quality through diameter control. We introduce tailored strategies for both random and cluster-based partitions for different application scenarios.

Extensive Experiment Analysis (§ 6). We conduct comprehensive experiments on real-world datasets for multi-index merging. We integrate our framework into popular graph-based indexes, including HNSW, NSG, SSG, and τ-MNG, demonstrating its broad applicability. Our method delivers up to 9.92× speedup while preserving search quality and efficiency. Moreover, the speedup ratio remains stable as the number of partitions increases with 50 sub-indexes and 100 million data, indicating robust scalability.

Due to space constraints, some additional proofs and experiments are omitted, which can be found in the Appendix.

2 Preliminaries

We first formally define the AKNN search problem in § 2.1 and then explain the graph-based AKNN index in § 2.2.

2.1 The AKNN Search Problem

Table 1 lists the frequently used notations and their definitions. We first describe the KNN search problem:

Definition 2.1 (KNN Search). Given a dataset X with n vectors in the D-dimensional Euclidean space $\mathbb{R}^D$, and a query vector $q \in \mathbb{R}^D$. Let $\delta(x, y)$ denote the distance between two D-dimensional vectors x and y. The KNN search retrieves k vectors KNN(q) in X such that $\delta(x, q) \leq \delta(x', q), \forall x \in \text{KNN}(q)$ and $x' \notin \text{KNN}(q)$.

With the rapid growth of dataset dimensionality and scale, exact KNN search fails to meet the efficiency requirements, known as the curse of dimensionality [28]. Therefore, most efforts have focused on approximate AKNN search, which trades a small loss in accuracy for a significant gain in efficiency. Recall@K is commonly used as a metric to evaluate the precision of the AKNN search algorithm. Recall@K is defined as the ratio of the true k nearest neighbors that appear in the k nearest results returned by the AKNN search algorithm, and it is computed as follows:

$$\text{Recall@K} = \frac{|\text{AKNN}(q) \cap \text{KNN}(q)|}{k} \tag{1}$$

where AKNN(q) is the top-k nearest results returned by the AKNN search, and KNN(q) is the actual KNN for a given q. Based on the above definition, numerous AKNN Search Indexes have been proposed with the aim of achieving higher recall at a lower cost.

Algorithm 1: AKNN Beam Search

Input: Graph Index $G = (V, E)$, Entry Points $\{ep\}$, Query q, Search Effort ef, Result Number k
Output: Top-k approximate nearest neighbors of q

```
1  Initialize priority queue C ← ∅;
2  C ← {(ep, δ(ep, q))};
3  Each node v ∈ V set v.expanded ← false;
4  while C has unexpanded nodes do
5      u ← nearest unexpanded node in C; u.expanded ← true;
6      for each unexpanded neighbor v of u do
7          C ← C ⋃ {(v, δ(v, q))};
8      if |C| > ef then
9          resize C to size ef;
10 return k nearest nodes in C;
```

2.2 Graph-Based AKNN Search Index

Most AKNN search algorithms find the approximate nearest neighbors via well-designed indexes, which can be divided into four categories: hash-based [13, 18, 18, 23, 49, 54–56, 64, 65], quantization-based [2, 3, 6, 19–22, 31, 69, 70], tree-based [8, 12, 15, 43], and graph-based [4, 9, 16, 17, 24, 26, 32, 39, 40, 45, 52, 53, 58, 61, 80]. Among them, the Graph-based methods have gained prominence due to their state-of-the-art search efficiency. Specifically, the Graph-based AKNN indexes take each vector in X as a node on the graph and construct a Proximity Graph (PG) to represent the similarity relationships between vectors. Then, given a dataset X with n vectors in the D-dimensional Euclidean space $\mathbb{R}^D$, we construct a graph index $G = (V, E)$ for X with vertex set V and edge set E: Every $v \in V$ corresponds to a unique vector x in X, and an edge $(u, v) \in E$ denotes the neighbor relationship between $u, v \in V$.

Most graph-based methods utilize a beam search algorithm presented in Algorithm 1 to retrieve AKNNs. During the beam search, set C is maintained as the candidate nearest neighbors of q, and its size ef is used to measure the search effort (line 1). The beam search starts from the given entry points (line 2). At each step of the search, it checks the neighborhood of the nearest unexpanded node and tries to insert these neighbors into C (lines 4-9). The algorithm terminates when no unexpanded node can be found in C (line 10).

The PG index is constructed in two stages: (1) Perform beam search in Algorithm 1 on the current index and obtain neighbor candidates W from top-$ef_{construction}$ ($ef_{construction} \gg ef_{search}$) results [39, 40] or visited nodes during searching [17, 45, 53]. (2) Apply the neighbor pruning rule of RNG (Relative Neighbourhood Graph) [43, 57] or its variants [16, 36, 45, 47, 53, 72] to W to select neighbors. Reverse edges are further added for better connectivity.

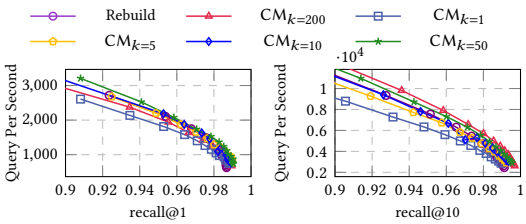


Figure 3: Nearest Neighbors being Critical Factor

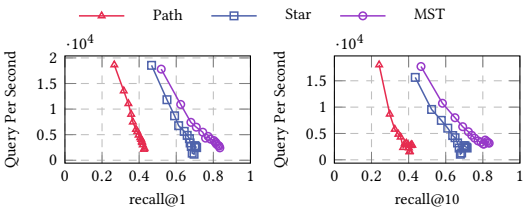


Figure 4: Search Performance of Different Merge Order

3 PROBLEM ANALYSIS AND MOTIVATION

In this section, we formalize the multi-index merging problem and present key observations that motivate our method (§ 3.1). We then introduce the pipeline of our merging framework (§ 3.2).

3.1 Problem Analysis and Motivation

We begin our analysis by defining the different partitions to merge. We consider m disjoint partitions $\{X_1, X_2, \ldots, X_m\}$ of a dataset X, where $X = \bigcup_{i=1}^{m} X_i$ and $X_i \cap X_j = \emptyset$ for all $i \neq j$. A **random partition** consists of equal-sized subsets formed by randomly assigning vectors, whereas a **cluster partition** consists of subsets with different sizes reflecting clustered (*i.e.,* kmeans) data. This disjoint-partition setting simplifies the formulation, while real-world updates may produce more complex scenarios such as overlapping indexes. However, as discussed in **§ 1**, search-based methods also support merging overlapping indexes with unique vector IDs. Given m random/cluster partitions $\{X_1, X_2, \ldots, X_m\}$ following the above definition, the m-graph index merge problem is defined as follows:

Problem Definition. Given m well-constructed graph indexes $I = \{G_1, G_2, \ldots, G_m\}$, where $G_i = (V_i, E_i)$ and $V_i = X_i$, the m-graph index merge problem aims to construct a unified graph index $G = (V, E)$ that satisfies the following constraints:

- *Search Quality*: The merged graph G achieves comparable search performance to an index built from scratch on X.
- *Computational Efficiency*: The merging of multiple indexes requires less time than reconstructing an index for X from scratch.

The straightforward index merging paradigm ***Cross-query Merge (CM)*** is based on pairwise searching: For each point x in a source index X_i, it queries all $m-1$ target indexes $I \setminus \{G_i\}$ to retrieve a candidate neighbor set W of size ef_c, which corresponds to the construction parameter. However, since it is unknown which indexes contain the top-ef_c candidates, all $m-1$ target indexes must be searched, incurring $m-1$ times ef_c-NN searches and $(m-1) \cdot ef_c$ candidates in total. As we have $|V|$ points to merge, it requires $O(|V|(m-1))$ searches with $O(|V| \cdot (m-1) \cdot ef_c)$ intermediate candidates, which is both time-consuming and could even take longer time than reconstruction. Obviously, reducing $m-1$ and ef_c could reduce this large complexity. In the following, we provide two observations on how we could reduce them without sacrificing index quality.

Observation 1: Nearest Neighbors are Key Contributing Components of Merging. As mentioned above, most graph-based methods construct proximity graphs using a two-stage framework, where higher-quality pruning candidates W lead to better index quality. These pruning candidates W aim to cover a large region around a given vertex, which directly leads to a high time cost. W can be divided into two parts: (1) Nearest neighbors that provide good connectivity among KNNs. (2) Distant neighbors that enable efficient graph traversal by introducing long-range edges and covering broader regions of the data space [74]. We find that **nearest neighbors are critical for merging quality**, as the distant neighbors are already captured within sub-indexes. This phenomenon can be verified in the following example:

Example 1. *In this example, we compare the search performance of the rebuilt index with the merged index from sub-indexes of 2 equal-sized disjoint random partitions of the SIFT1M dataset. We first construct an HNSW index directly on the complete dataset with $ef_c = 200$ and denote it as **Rebuild**. Then we relax the per-target-index search effort of CM from the fixed ef_c to $k = [1, 5, 10, 50, 200]$, where 200 corresponds to the default setting. CM_k indicates that the top-k results from the search are used for the merge of the 2 sub-indexes, so $k = 1$ keeps only the nearest neighbor. As illustrated in Fig 3, we investigate the impact of k from target indexes on merged index quality. Specifically, incorporating nearest neighbors from other indexes dramatically improves performance even with just $k = 1$, and it achieves the identical performance of **Rebuild** at high recall (≥ 0.99). With larger k from target indexes, the merged index achieves a better efficiency-accuracy tradeoff.*

Based on this observation, we can safely reduce the merging problem to a more efficient formulation. Standard index construction typically requires large pruning candidate sets with $|W| \geq 200$. In contrast, for merging purposes, we only need to identify the top-k nearest neighbors from the target index for each vertex. Here, k can be substantially smaller, *e.g.,* $k \in [1, 50]$, which saves the computation significantly by limiting the search to a small number of nearest neighbors from other indexes, making the merging process substantially more efficient with smaller k.

Observation 2: Different Merge Orders Lead to Varying Search Performances. The merge order specifies which pairs of indexes to merge next. According to our analysis, retrieving KNNs from other indexes for each vertex is critical, which naturally leads to a pairwise processing framework. However, this framework suffers from inherent computational redundancy for the following reasons: (1) KNNs from different partitions may occlude one another during merging. Empirically, only 9.2% of the 100-NNs remain as neighbors of the vertex after neighbor selection. (2) For cluster partitions, KNNs of a vertex are often concentrated in a subset of partitions rather than uniformly distributed across all partitions. Moreover, we observe that **even with the same number of merge operations, different merge orders result in significantly different search performance,** as shown in the following example:

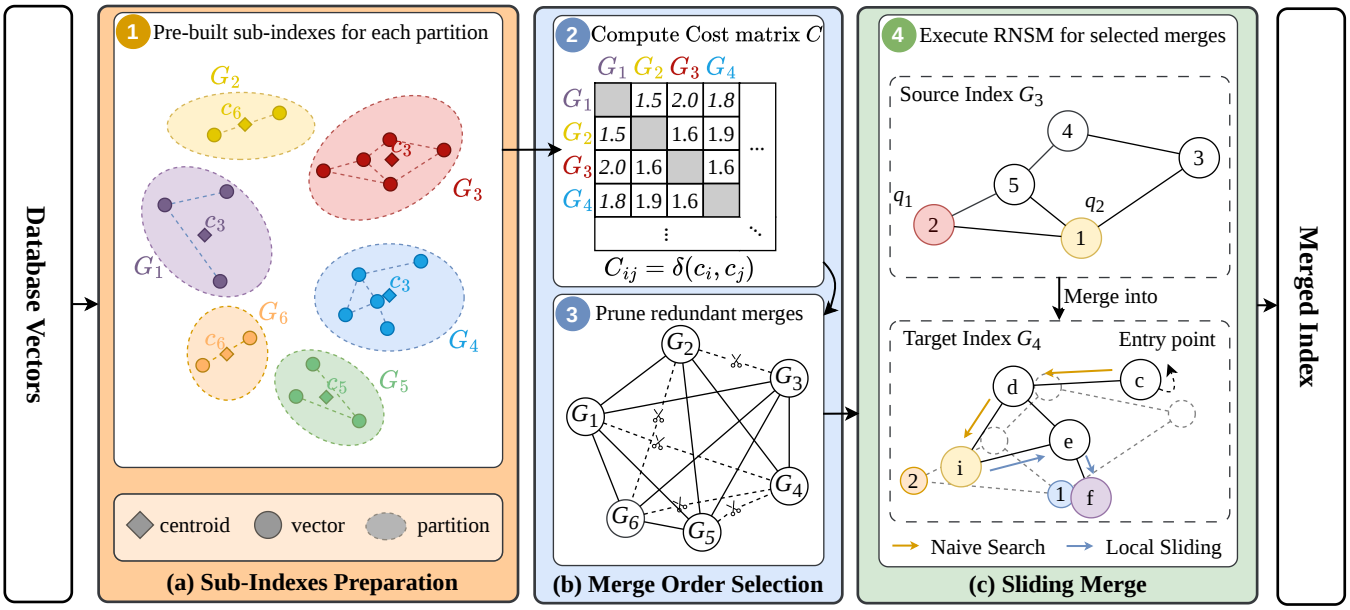


Figure 5: Pipeline of Our Multi-Index Merge Framework

Example 2. *We compare ten random partitions of SIFT1M merged with the same CM pairwise algorithm under three merge orders, isolating the effect of the merge order.* ***Path*** *forms a merging chain,* ***Star*** *merges all indexes with a central one, and* ***MST*** *builds a minimum spanning tree based on inter-index similarity. All three orders use the same $m-1$ merge operations. As shown in Fig 4,* ***Path*** *yields low recall because cross-partition KNNs may lie at opposite ends of the chain, while* ***Star*** *and* ***MST*** *improve cross-partition reachability. Moreover,* ***Star*** *underperforms* ***MST*** *because its central hub incurs an imbalanced merge workload.*

3.2 Multi-index Merging Framework Overview

Fig 5 illustrates the pipeline of our proposed RNSM + MOS framework, which has the following three stages:

a) Sub-Indexes Preparation. While existing methods are tailored to either cluster partitions [53, 74] or random partitions [66, 68], our framework can be applied to both partition types without compromising performance. Given pre-built sub-indexes, we first prepare the centroids of each partition. Then the partition centroids are utilized for merge order selection to compute the cost matrix C.

b) Merge Order Selection (MOS). Next, we reduce the cost of multi-index merging through MOS. It acts as a lightweight merge planner on top of pairwise RNSM. We formulate MOS as selecting a connected merge order graph that minimizes the total cost of the selected pairwise merges, subject to degree and diameter bounds that limit total merge workload and cross-partition traversal length, respectively. We also use centroid distances to prioritize the merge of closer partitions, as nearby indexes are more likely to share KNNs and thus benefit more from merging [1]. Specifically, we first compute pairwise centroid distances to form a cost matrix C. For random partitions, we prove they are identical in expectation and use a constant. For cluster partitions, such pairwise centroid distance computation cost is negligible given that the partition number is much smaller than the dataset size. Then, we construct a sparse merge order graph that effectively prunes unnecessary merges (*e.g.*, G_2 and G_3) according to C and the requirements of degree and diameter.

c) Sliding Merge. Finally, we execute each edge selected by MOS as an RNSM operation following two common principles: merge smaller indexes into larger ones [60], and merge closer partitions first [1]. RNSM exploits prior search results to accelerate later searches. Specifically, we greedily select nodes with the maximum number of Reverse k-Nearest Neighbors (RKNN) in the source index as pivots. RKNN of a vector x is a vector that regards x as one of its KNNs. Then, the search result i of pivot 2 is used as an entry point to find cross-index neighbors for RKNNs of pivot 2.

4 RNSM for Pairwise Indexes Merge

This section details the proposed Reverse Neighbor Sliding Merge framework for merging two graph indexes. We first formally define the concept of *sliding merge* and derive a corresponding cost model in § 4.1 to quantify the merging overhead. Then, we introduce the RNSM algorithm in § 4.2, demonstrating how it accelerates the merging process by reducing the derived cost objective.

4.1 Sliding Merge Cost Model

Our preliminary analysis identifies the AKNN search within the target index as the primary bottleneck in index merging (see Fig 3). Consequently, optimizing the search process on the target index is paramount. Prior research established that initializing graph-based searches with entry points topologically closer to the query target significantly reduces the search path length, thereby reducing latency [17, 53, 79]. Building on this insight, we propose *Sliding Merge*, a strategy that leverages search results from neighboring nodes to accelerate processing. In this section, we formalize the cost model and define the optimization problem.

Notation and Sliding Semantics. Given a source graph index $G_1 = (V_1, E_1)$ and a target index $G_2 = (V_2, E_2)$, our objective is to merge G_1 into G_2 by performing an AKNN search in G_2 for every vector $x \in V_1$. The *Sliding Merge* framework employs two distinct search operations:

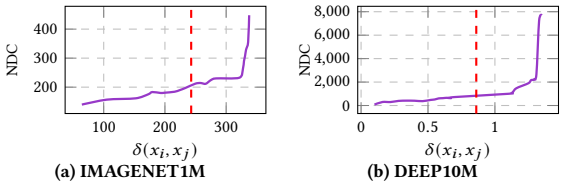
(1) **Naive Search:** A standard AKNN search for query $x \in V_1$ starting from the default entry point of G_2.
(2) **Local Sliding:** A search for query x initialized using the result of a previous search for a pivot node p.

Let $\mathcal{L}(p, x)$ denote the search cost for query x. If a pivot p is used, the cost depends on the proximity between p and x; otherwise, a fixed base cost is incurred.

Cost Metric. To ensure our model is hardware-agnostic, we employ the Number of Distance Computations (NDC) as the cost metric, a standard practice in nearest neighbor search analysis [20, 40, 53, 76, 81]. We define the cost of a *Naive Search* as a constant γ, derived from the average NDC required for a search from the default entry point to reach the prescribed recall level, like 0.98. However, estimating γ from target recall is impractical in real systems because of the non-existent ground truth. We therefore use a sufficiently large naive-search effort ef_N (a common graph index setting [62]) to ensure reliable pivot searches. Thus, γ is used for formalization while ef_N is the practical parameter. Moreover, our experiment shows a strong linear correlation ($R^2 = 0.997$) between search effort ef and NDC. We therefore approximate the NDC of a search with effort ef as $\alpha \cdot ef$, where α is the empirically fitted NDC–ef slope. Accordingly, the Naive Search cost is approximated as $\gamma \approx \alpha \cdot ef_N$. For *Local Sliding*, recent findings [67] indicate that when a query x starts from a pivot p proximate in the high-dimensional space, the NDC required to converge follows a linear relationship with their distance $\delta(p, x)$ (see Fig 6). We measured the 95th percentile of the k-th NN distance and found that the top-500 NNs empirically fall within this linear range. Since RNSM performs local sliding for top-3~10 NNs, we can approximate the sliding NDC cost as $\mathcal{L}(p, x) \approx \beta \cdot \delta(p, x)$, where β is the empirically fitted distance–NDC slope. By matching the NDC with $\alpha \cdot ef = \beta \cdot \delta(p, x)$, we can estimate the search effort ef_L for sliding from p to x as $ef_L = \lambda \cdot \delta(p, x)$, where $\lambda = \beta / \alpha$.

Problem Formulation. While local sliding offers lower costs than naive search (see Fig 6), it necessitates a prerequisite: the pivot p must have already been searched. Ideally, every node would slide from its nearest resolved neighbor. However, identifying the optimal global pivot for x would itself require a costly search. To make this tractable, we exploit the topology of the source graph G_1. Since G_1 is a proximity graph, neighbors in G_1 are inherently close in the vector space [17]. Thus, we restrict the candidate pivots for a node v to its immediate neighbors $N(v)$ in G_1. This transforms the merge optimization into a pivot selection problem: we must select a subset of nodes to perform Naive Search (pivots), such that all other nodes can perform low-cost Local Sliding from a pivot. Moreover, we require pivots to be independent of each other to fully decouple them. We formally define this as follows:

Definition 4.1 (Minimum Cost Independent Dominating Pivot Selection (IDPS)).



(a) IMAGENET1M (b) DEEP10M

Figure 6: Verification of Distance Cost Model. In datasets IMAGENET1M and DEEP10M, the cost of sliding from x_i to x_j is linear to the distance when x_i and x_j are close[67]. The red dashed lines mark the 95th percentile distance of the top-500 nearest neighbors in each dataset.

- **Input:** A vector dataset X, a graph $G = (V, E)$ where node $v_i \in V$ corresponds to $x_i \in X$, a distance function $\delta(\cdot, \cdot)$, and a naive search cost γ, distance-NDC slope β.
- **Output:** A pivot subset $P \subseteq V$ that minimizes the total cost $\mathcal{L}(P)$, subject to the independent dominating set constraint.
- **Constraint:** Every node $v \in V$ must be either a pivot or covered by a pivot:

$$v \in P \vee \exists p \in P \text{ s.t. } (v, p) \in E$$

  Additionally, P is an *independent set*: no two pivots are adjacent in G.
- **Objective:** Minimize $\mathcal{L}(P) = \mathcal{L}_{naive}(P) + \mathcal{L}_{slide}(P)$, where:

$$\mathcal{L}_{naive}(P) = \gamma \cdot |P|$$
$$\mathcal{L}_{slide}(P) = \sum_{v \in V \setminus P} \min_{p \in P \cap N(v,G)} \beta \cdot \delta(x_v, x_p)$$

The independence constraint (no pivot slides from another pivot) is a conservative design choice for parallel execution rather than a guarantee of the minimum pivot count. Although allowing adjacent pivots can sometimes reduce the number of Naive Searches, reusing search results along an adjacent-pivot chain introduces multi-hop sliding and sequential dependencies, exemplified by the MST-based propagation in SimJoin [67]. Such dependencies limit parallel scalability, as shown in Fig 15. We next show the computational hardness of this IDPS problem.

Theorem 4.2. *The Minimum Cost Independent Dominating Pivot Selection (IDPS) problem is NP-hard.*

Proof Sketch. We reduce from the NP-complete Minimum Independent Dominating Set (MIDS) problem [30]. Given a MIDS instance $\langle G', k \rangle$, set $G = G'$, define the discrete metric $\delta \equiv 1$ for distinct nodes, and set $\gamma = 2$, $\beta = 1$, $C = k + |V|$. For any feasible P (an independent dominating set), the cost simplifies to $\mathcal{L}(P) = 2|P| + (|V| - |P|) = |P| + |V|$. Hence $\mathcal{L}(P) \leq C$ iff $|P| \leq k$, establishing a one-to-one correspondence between MIDS solutions and IDPS solutions. The full proof is provided in the Appendix. □

Given the intractability of the IDPS problem, we resort to a greedy algorithm for pivot selection presented in Algorithm 2.

4.2 Reverse Neighbor Sliding Merge

A straightforward algorithm for IDPS is to adopt a greedy strategy: iteratively selecting the pivot with the highest *unit value* (coverage nodes divided by cost). However, applying this directly to index

merging is problematic for two reasons. 1) *Limited Visibility*: Relying solely on the sparse connections in the source PG limits the ability to find the true nearest pivot, potentially forcing sub-optimal sliding paths. 2) *Computationally Expensive*: Since pivots and their sliding nodes are not necessarily adjacent in the source index, calculating unit values requires either exhaustive all-pairs distance computation or reliance on graph indexes that often store neighbor rankings rather than explicit distance values [67], compromising both flexibility and precision.

The RNSM Strategy. To overcome these limitations, we propose *Reverse Neighbor Sliding Merge (RNSM)*. The core intuition is to resolve pivot selection *globally* before merging, enabling greater parallelism through independent pivots. It uses RKNN relations to identify pivots: if $x \in N_k(y)$, then x is inherently a good pivot for y due to proximity. Consequently, nodes with many RKNNs are natural hubs that can serve as cost-efficient pivots for many neighbors. We first introduce two core components of the RNSM: Pivot Selection and Local Sliding.

Pivot Selection. We first identify the pivots and their corresponding followers to perform local sliding before executing the merge. Each node in the source index G_1 is classified into:

(1) **Pivots** $p \in P$**:** Neighbor candidates obtained by a naive search in target index G_2.
(2) **Follower set** F_p**:** Neighbor candidates obtained by local sliding from search results of pivot p.
(3) **Uncovered set** U**:** The remaining nodes.

As shown in Fig 6, for the search results of one node to serve as effective entry points for another node, the two nodes must lie in proximity in the metric space. This motivates us to select nodes that are close to many other nodes as pivots. We observe that RKNNs naturally capture this notion: unlike KNNs, the exact number of RKNNs of a node is not known in advance. Nodes with more RKNNs are ideal pivot candidates, serving as the nearest neighbors of many others. To be specific, we first expand the neighborhood of each node $v \in V_1$ by searching from v itself for its KNNs $N_k(v)$ (nearest neighbors expansion). With $N_k(v)$ for each node v, we construct their RKNNs $R_k(v)$. Let $F_v = R_k(v) \cap U$, we define the gain of selecting v as pivots to be:

$$g(v) = \frac{|F_v|}{\sum_{u \in F_v} \delta(x_v, x_u)}$$

For simplicity, we apply a uniform search effort ef_L to all followers by taking the maximum possible distance of any pivot to its follower (see *Local Sliding* below). The gain $g(v)$ thus simplifies to $|F_v|$, and the IDPS problem reduces to exactly the Minimum Independent Dominating Set (MIDS) problem. Since MIDS is inapproximable within $|V|^{1-\epsilon}$ for any $\epsilon > 0$ unless P = NP [25, 30], we use a lazy greedy algorithm. Specifically, it maintains a max-heap ordered by $g(v)$ and greedily selects the node with the maximum gain as a pivot, assigning its uncovered RKNN members as followers, until $U = \emptyset$. Since $g(v)$ can only decrease as U shrinks, the stored value in the heap always upper bounds the true gain. We exploit this monotonicity via lazy evaluation: a node popped from the heap is selected immediately if its gain is still current, and reinserted with the updated gain otherwise, avoiding redundant recomputation.

Algorithm 2: Reverse Neighbor Sliding Merge (RNSM)

Input: Source Graph $G_1 = (V_1, E_1)$, Target Graph $G_2 = (V_2, E_2)$, Expansion Factor k^+, Reverse k, Naive Search ef_N, Local Sliding ef_L
Output: Merged Graph $G = (V, E)$

1 $V \leftarrow V_1 \cup V_2$; $E \leftarrow E_1 \cup E_2$;
2 $P \leftarrow \emptyset$; // Set of Pivots
3 $Covered \leftarrow \emptyset$; // Set of merged nodes
/* Phase 1: Pre-computation */
4 $G_1' \leftarrow$ **ExpandNeighbors**(G_1, k^+);
5 $\{R_k(x) \mid \forall x \in V_1\} \leftarrow$ **BuildReverseIndex**(G_1', k);
/* Phase 2: Pivot Selection (Lazy Greedy) */
6 Initialize max-heap H with $(|R_k(x)|, x)$ for all $x \in V_1$;
7 **while** $H \neq \emptyset$ **do**
8 $(g, x) \leftarrow$ uncovered x with maximum gain;
9 **if** *updated gain* $g' < g$ **then**
10 ReInsert(g', x); **continue**;
11 $F_x \leftarrow R_k(x) \setminus Covered$;
12 $P \leftarrow P \cup \{x\}$; $Covered \leftarrow Covered \cup \{x\} \cup R_k(x)$;
/* Phase 3: Parallel Merging */
13 **for** $p \in P$ ***in parallel*** **do**
// Pivot: Naive Search
14 $Res_p \leftarrow$ SearchGraph$(G_2,$ query $= p,$ entry $=$ default, $ef_N)$;
15 UpdateGraph(G, p, Res_p);
// Followers: Local Sliding
16 **for** $y \in F_p$ **do**
17 $Res_y \leftarrow$ SearchGraph$(G_2,$ query $= y,$ entry $= Res_p, ef_L)$;
18 UpdateGraph(G, y, Res_y);
19 **return** G;

Local Sliding. Next, with each node classified into pivots and followers, we start efficient merging with local sliding. Specifically, each pivot performs a naive search from the default global entry point of G_2, which requires a larger search effort ef_N to compensate for the potentially large distance to the target region. Once the naive search for pivot p completes, all followers in F_p proceed to local sliding using the naive search results Res_p as entry points. Since each follower $v \in F_p$ has pivot p as one of its KNNs in G_1, Res_p in G_2 are likely to be close to the true neighbors of v, as the neighbors of a neighbor are likely neighbors [14]. This yields high-quality entry points for local sliding, allowing a much smaller ef_L to suffice. In practice, we use the same ef_L for all slidings, corresponding to $ef_L = \lambda \cdot \max_{v \in P, u \in F_v} \delta(x_v, x_u)$. Compared to naive search for all nodes with effort ef_N, local sliding approximately reduces the total search cost by

$$(1 - |P|/|V_1|) \cdot (1 - ef_L/ef_N)$$

Finally, we update the merged graph by applying heuristic edge selection to the search/sliding results. For each source node, its candidates are concatenated with its existing neighbors and pruned together. The target index nodes acquire cross-partition connections solely via reverse edges inserted during this process, without performing explicit searches of their own. Empirically, this asymmetric strategy incurs negligible quality loss while halving the merge cost [33, 40]. The details of RNSM are presented in Algorithm 2.

(1) **Neighbor Expansion (Line 4):** We first densify the source graph G_1. Existing PGs are sparse approximations of the true KNN graph. We expand the neighborhood of each node x to a larger size k^+ by traversing the graph. This creates a richer set of candidate connections, ensuring that nodes can access their true nearest pivots, aligning with the IDPS objective.

(2) **Reverse Inversion (Line 5):** We construct the RKNN set $R_k(x) = \{y \mid x \in N_k(y)\}$ for every node with a relative small k (*i.e.,* 3,5,7).
(3) **Pivot Selection (Lines 6-12):** We maintain a max-heap ordered by marginal gain $|R_k(x) \setminus Covered|$. At each step, we extract the top candidate; if its stored gain is stale, we reinsert it with the updated gain (lazy evaluation). Once the gain is confirmed current, we select x as a pivot and assign all uncovered $R_k(x)$ to slide from x.
(4) **Parallel Execution (Lines 13-18):** Finally, we execute the merge in parallel. Pivots perform a *Naive Search* on G_2. Their search results become the entry points for the *Local Sliding* of their followers.

It is worth noting that both **Reverse Inversion** and **Pivot Selection** account for a negligible fraction of the total merge time (typically less than 5%), as the merge process is compute-intensive and dominated by distance computations during **Neighbor Expansion** (≈ 10%) and **Parallel Execution** (≈ 85%). A detailed runtime breakdown of each RNSM phase is provided in Appendix Exp-a.

Example 3. *As illustrated in Fig 7, RNSM merges source index G_1 into target index G_2 in four steps. Nearest neighbor expansion in Fig 7(a) first obtains AKNNs for each node in G_1, which is a cheap beam search starting from the nodes themselves. Next, in Fig 7(b), given the AKNNs, we get the RKNNs of each node in G_1 and rank them by the number of RKNNs for pivot selection. Then, nodes 5 and 8 are greedily selected as pivots for their higher number of RKNNs, while node 4 is excluded since it has already been assigned as a follower of pivot 5. In Fig 7(c), we finally start sliding merge on the target index with node 8 and its followers {3, 9, 10} as an example. Node 8 first performs a naive search from a distant initial entry point b in G_2 and retrieves i as an ANN. Then, its followers use i for local sliding. Because they are close to pivot 8, their nearest neighbors {g, f, j} lie near i and are reached with fewer hops.*

4.3 Performance Analysis

Advantages of RNSM. Using RKNNs yields several structural benefits: (1) **Natural Hub Identification:** The metric $|R_k(x)|$ identifies density peaks (hubs) purely from graph topology without requiring expensive clustering or centroid computations. (2) **Dynamic Load Balancing:** Unlike fixed k approaches, RNSM allows variable cluster sizes. Dense regions generate pivots with large $|R_k(x)|$, amortizing the naive search cost over more nodes, while sparse regions degenerate gracefully. (3) **Cost Reduction:** By selecting pivots with the most extensive coverage first, we greedily reduce the number of required pivots $|P|$, directly reducing the naive search term in our cost model. The independent-set constraint further boosts parallel-friendly execution.

Complexity Analysis. We analyze the time complexity of merging a source graph index G_1 into a target graph index G_2 to produce a unified index $G = (V, E)$, where $n = |V|$ denotes the total number of vertices. Without loss of generality, we assume the source index is not larger than the target index (*i.e.,* $|G_1| \le |G_2|$). If initially $|G_1| > |G_2|$, the two are swapped so that we always merge the smaller structure into the larger one—a common optimization that speeds up merging. A key property of these graph-based indices, established in prior work [17, 29], is that the maximum node degree is bounded by a constant. This sparsity guarantee is crucial for achieving an efficient merge procedure. The merging process consists of three sequential stages:

1. Neighbor Expansion: Expanding neighbors involves an incremental search with the graph index. Start with the nearest neighbor; the incremental cost of searching the top-k^+ neighbors can be expressed as $O(k^+)$ with a small constant factor [71]. This efficiency stems from interpreting the existing proximity graph (PG) index as a compressed representation of the k-nearest neighbor graph [17]. *Total Complexity:* $O(n \cdot k^+)$.

2. Pivot Selection: Constructing the RKNN graph requires iterating over all edges in the expanded graph. Given that the total number of edges is $n \cdot k^+$, the inversion step takes $O(n \cdot k^+)$ time. Subsequently, sorting the nodes on the RKNN graph using bucket sort can be performed in $O(|V| + |E|)$ time. Since $|V| = n$ and $|E| = n \cdot k < n \cdot k^+$, the *total complexity* of the entire process remains $O(n \cdot k^+)$.

3. Sliding Merge: Let $|P|$ be the number of selected pivots.

- *Pivots:* Perform naive AKNN search. In typical graph structures (NSG, HNSW), the search complexity is bounded by $c_{naive} = O(\log n \cdot ef \cdot d)$, where ef is the search parameter and d is the average degree. Total: $O(|P| \cdot c_{naive})$.
- *Followers:* Perform local sliding. Since the pivot $p \in N_k(y)$ by construction, the sliding path length is proportional to the distance $\delta(p, y)$ and is small. In practice, the search complexity is bounded by a small constant c_{slide}. Total: $O((n - |P|) \cdot c_{slide})$.

Overall Complexity: Both local sliding and naive search have a higher cost than neighbor expansion, where $c_{naive} > c_{slide} > k^+$. The overall complexity can be derived to $O(|P|c_{naive} + (n-|P|)c_{slide})$. Since $|P| < n$ (depending on k, typically around 20% of nodes), the dominant term is reduced from $n \cdot c_{naive}$ (all-naive search) to roughly $n \cdot c_{slide}$, representing a significant speedup.

5 MOS for Multiple Indexes Merge

In this section, we minimize the multi-index merge cost by reducing redundant operations. We first formalize the merge order selection task. Then, we propose algorithms for our formalized problem.

Merge Order Selection. Recall that merging requires finding the KNNs in all other indexes for every node in each source index. Naive approach requires $O(m^2)$ index merge operations between m indexes. However, this approach contains significant redundancy in practice. Besides, our empirical analysis (Fig 4) shows that different merge order strategies yield substantially different search performance. Thus, we formally define the task as follows: Given m dataset partitions $\{P_1, P_2, \ldots, P_m\}$, we represent partitions as vertices $V = \{v_1, \ldots, v_m\}$ and merge operations as edges E, forming a merge order graph $\mathbb{G} = (V, E)$. Each edge $(v_i, v_j) \in E$ indicates that we perform index merging between index G_i and G_j. The merge order selection (MOS) task aims to adjust the merge order graph $\mathbb{G}$ to achieve better individual sub-index connections at low cost.

Cost Model. Given the notation above, our objective is to extract a subgraph from the complete graph of pairwise merges. We aim to identify a merge order—defined by this subgraph—that achieves search performance comparable to either a full index rebuild or

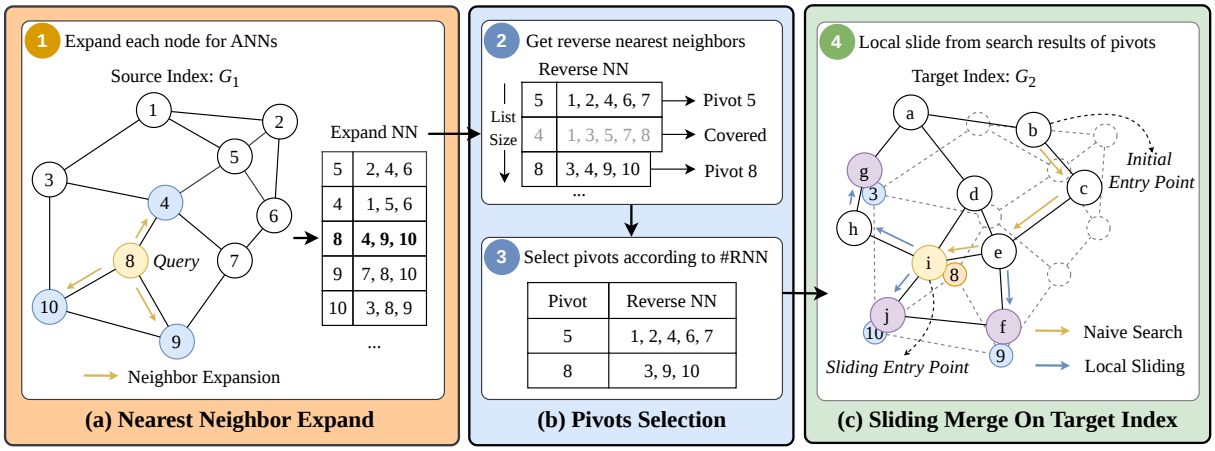


Figure 7: Example of Two-Index Merging with RNSM

an exhaustive pairwise merge strategy. We first prioritize merges using centroid distance, since closer partitions are more likely to share KNNs [1]. For random partitions, the exchangeability of partition labels implies identical expected centroid distances for all partition pairs. Therefore, we assign a unit merge cost to every pair. For clustered partitions, we directly compute the centroid distances for all the partitions. Moreover, § 3.1 indicates that different merge order strategies can lead to substantially different search performance. MOS therefore does not rely solely on a proximity-based centroid distance proxy. Instead, it combines it with two additional proxies: degree and graph diameter. Degree R is introduced for merge efficiency, since it controls the number of pairwise merges. Experimentally, a high-degree partition may receive an overwhelming number of new out-neighbor candidates and hurt quality, as shown in *Star* of Fig 4. Diameter Δ is introduced for global search quality, given that a large diameter may reduce recall, as shown in *Path* of Fig 4. This is because the diameter indicates the maximum number of intermediate partitions needed to reach a cross-partition nearest neighbor. We now formally define the MOS problem with these proxies:

MOS Problem. Therefore, we propose extracting subgraphs from multiple perspectives with minimal cost, specifically:

Definition 5.1 (Minimum Cost Merge Order Selection (MOS)).

- **Input:** Given a set of m vertices $V = \{v_1, v_2, \ldots, v_m\}$ and a symmetric cost matrix C where C_{ij} is the cost of connecting vertex v_i and v_j, diameter bound Δ and degree bound R.
- **Output:** An undirected graph $\mathbb{G} = (V, E)$ such that:
- **Constraint:** (1) Connectivity: The graph $\mathbb{G}$ is connected. (2) Maximum Degree: For all vertices $v \in V$, the degree $\deg(v) \leq R$. (3) Graph Diameter: The diameter of $\mathbb{G}$ is at most Δ.
- **Cost Objective:** Minimize the total cost of the graph:

$$\min \sum_{(v_i,v_j)\in E} C_{ij}$$

The constraints serve distinct objectives:

(1) **Connectivity:** This is the fundamental completeness requirement, ensuring that the search space is fully traversable.

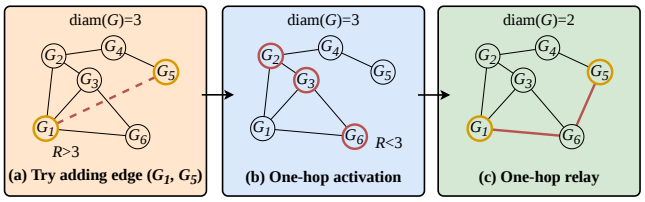


Figure 8: Example of Merge Order Selection

(2) **Degree Bound (R):** This constraint is merge efficiency and parallel-friendly design. Inserting new edges locks a large proportion of proximity graph nodes, so excessive degree serializes updates and causes parallel workload imbalance.

(3) **Diameter Bound (Δ):** This constraint governs the worst-case search latency. The diameter determines the maximum number of intermediate index hops required to traverse between any two partitions.

While minimizing the total edge weight reduces the direct merge costs, the graph edge weights also affect the runtime performance as shown in our empirical analysis in § 4.3. Ignoring the diameter bound reduces the problem to the NP-hard Bounded Degree Minimum Spanning Tree Problem [34]. To address this NP-hard problem efficiently, we propose a greedy heuristic as follows. We initialize the merge order graph as an empty structure and iteratively insert nodes to satisfy the Connectivity constraint. Subsequently, for each node, we establish connections with nodes located beyond Δ-hop distance (typically Δ=2) to ensure compliance with the Diameter Bound. When either node possesses R neighbors, we relay the connection to its $(\Delta - 1)$-hop neighbors to meet both the Degree and Diameter Bounds whenever a degree-feasible relay exists. Throughout this process, connections are prioritized according to the cost matrix, with lower-cost pairs being added first. The detailed MOS algorithm is presented below.

MOS Algorithm. Algorithm 3 introduces the detailed procedure of MOS. We begin with an empty graph $\mathbb{G}$ and iteratively add edges (lines 2-3). We process vertices $v \in V$ in ascending order of their average distance to the R nearest centroids, prioritizing centrally-located partitions (lines 4-6). For each v, we iteratively connect it to its nearest neighbor x that is more than Δ hops away (line 8). If

both v and x have degree less than R, we directly add edge (v, x) (lines 9-10). If exactly one vertex has reached the degree limit R, we employ a 1-hop relay strategy (lines 11-15), select a neighbor x' of the degree-saturated vertex, and connect it to the unsaturated one. Note that the subgraph may not exist due to the Moore bound [27]; however, we still enforce connectivity in practice for completeness of results.

Example 4. *As illustrated in Fig 8, MOS constructs a workload-balanced merge order graph with diameter 2 by inserting and relaying edges within 1-hop neighborhoods. Suppose we insert edge* (G_1, G_5) *when the degree limit of 3 is exceeded for* G_1 *(i.e.,* $\deg(G_1) = 4$*). We then relay this connection to one of the 1-hop neighbors of* G_1*, and* G_6 *is selected since* $\deg(G_6) < 3$*. By relaying edge* (G_1, G_5) *as* (G_5, G_6)*, we obtain a graph with maximum degree at most 3 and diameter 2.*

Given the merge-order graph constructed by MOS, we execute each selected edge as a pairwise RNSM operation according to two commonly adopted principles. First, we prioritize edges between closer partitions because nearby partitions are more likely to share nearest neighbors [1]. Second, for each selected pair, we merge the smaller sub-index into the larger one [33, 60]. Consequently, only vectors in the smaller source index require Naive Search or Local Sliding to obtain cross-partition edges, further reducing the merge time. During each pairwise operation, newly identified cross-partition candidates are concatenated with existing neighbors and jointly processed by the edge-pruning rule of the PG index to meet the maximum outdegree bound. Accordingly, when a sub-index is incident to multiple selected edges, the centroid-distance priority determines the next pair to process, while the partition sizes determine the merge direction.

Complexity. We now analyze the time complexity of Algorithm 3. Constructing the cost matrix requires pairwise distance computations between m D-dimensional centroids, resulting in $O(m^2 \cdot D)$. The ordering requires $O(m^2 \log R)$ for computing average distances of R nearest neighbors and sorting vertices. The main loop adds $O(m \cdot R)$ edges total, with each edge addition requiring $O(m \cdot R)$ time for BFS-based incremental hop-distance matrix updates, yielding a complexity of $O(m^2 R^2)$. Overall, the time complexity of our MOS algorithm is $O(m^2(R^2 + D))$, independent of dataset size n.

Since m, R, and D are typically much smaller than the dataset size n, MOS is substantially less expensive than rebuilding or merging graph indexes over all n vectors. Across our evaluated configurations, the end-to-end time cost of the MOS accounts for only approximately 0.1%–1% of the corresponding Rebuild time.

Algorithm 3: Merge Order Selection MOS

Input: Partition Centroids $\{c_1, c_2, \ldots, c_m\}$, Cost Matrix $C = \mathbf{0}$, Degree Bound R, Diameter Bound $\Delta = 2$
Output: Merge order graph $\mathbb{G} = (V, E)$

1 $C \leftarrow$ compute cost matrix;
2 $V \leftarrow \{c_1, c_2, \ldots, c_m\}$; $E \leftarrow \emptyset$;
3 $H \leftarrow$ distance matrix in hops, initialized to ∞; $Distances \leftarrow \emptyset$;
4 **for** $c_i \in V$ **do**
5 $Distances[c_i] \leftarrow$ average distance of R nearest centroids;
6 order $\leftarrow V$ sorted in ascending order of $Distances$;
7 **for** $v \in$ *order* **do**
8 **while** *has nearest vertex* x *with* $H[v][x] > \Delta$ **do**
9 **if** $deg(v) < R$ ***and*** $deg(x) < R$ **then**
10 $E \leftarrow E \cup (v, x)$;
11 **else if** $deg(v) < R$ ***and*** $deg(x) \geq R$ **then**
12 $x' \leftarrow \arg\min_{x' \in \text{Neighbor}(x), \deg(x') < R} C_{x',v}$; $E \leftarrow E \cup (v, x')$;
13 **else if** $deg(v) \geq R$ ***and*** $deg(x) < R$ **then**
14 $v' \leftarrow \arg\min_{v' \in \text{Neighbor}(v), \deg(v') < R} C_{v',x}$; $E \leftarrow E \cup (v', x)$;
15 **else**
16 $e \leftarrow (v, x)$; // feasibility fallback
17 Update H;
18 **return** G;

Table 2: Dataset Statistics

Dataset	Dimension	Base Size	Query Size	LID [7]
SIFT	128	1,000,000	10,000	19.4
DEEP	96	100,000,000	1,000	19.5
GIST	960	1,000,000	1,000	38.9
IMAGE	768	10,000,000	10,000	21.2
ANTON	1024	10,000,000	1,000	28.9
MARCO	1024	10,000,000	1,000	31.0

6 Experiment

6.1 Experiment Setting.

Datasets. We use six public datasets with diverse scales and types. Including datasets that are widely used in the benchmarks (SIFT, DEEP, GIST)[1] and datasets generated from the latest embedding models (IMAGE[2], ANTON[3], MARCO[4]). The statistics of those datasets are summarized in Table 2, where *Base Size* denotes the dataset cardinality and *Query Size* denotes the number of queries for testing. We also take several slices of distinct sizes from the original datasets to test the scalability of our framework. For example, we use DEEP1M to represent 1 million slices of the DEEP dataset, and use ANTON10M to indicate the 10 million scale in the experiment.

Methods. We compare the following index merge methods:
- *Rebuild*: Rebuild the complete proximity graph directly by incrementally inserting nodes into an empty graph.
- CM: Cross-query Merge (CM) using naive search. It uses a smaller candidate set $ef \ll ef_{construction}$ and a one-way search (only searches the target index, not bidirectional), which are the two key optimizations used in HNSW-merger [33].
- FGIM: Method for merging PG indexes and transforming it into a KNN graph merging and refining problem [66].
- Overlap: Overlapping-based method that merges indexes through duplicate data points [53].
- RNSM: Our Reverse Neighbor Sliding Merge (RNSM) method without Merge Order Selection (MOS).
- RNSM*/RNSM+: Our RNSM with MOS for random/cluster partitions.
- CM*/CM+: CM with pairwise order for random/cluster partitions.

Performance Metrics. We evaluate the merged graph index based on the following two primary metrics:

- **Search Performance:** We measure efficiency via Queries Per Second (QPS) and accuracy via Recall@K as defined in § 2. Consistent with standard practice, we report the average Recall@10 and QPS with the top-10 AKNN search over the query set.

[1] https://www.cse.cuhk.edu.hk/systems/hash/gqr/datasets.html
[2] https://huggingface.co/datasets/kinianlo/imagenet_embeddings
[3] https://huggingface.co/datasets/anton-l/wiki-embed-mxbai-embed-large-v1
[4] https://huggingface.co/datasets/Cohere/msmarco-v2.1-embed-english-v3

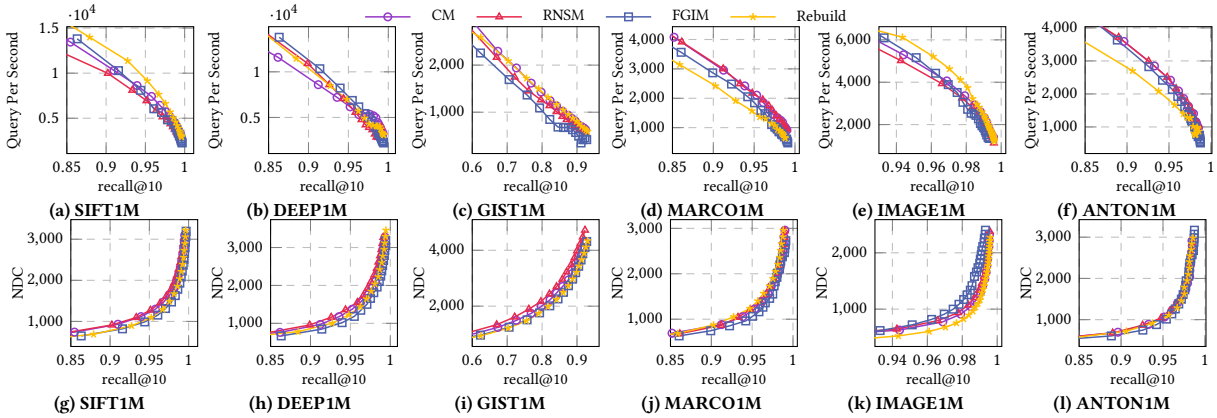


Figure 9: Recall@10–QPS and NDC Tradeoffs of Various Methods

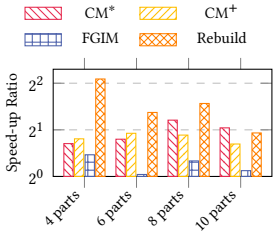


Figure 10: Speed-up Ratio Comparison across Different Partitions with MARCO10M

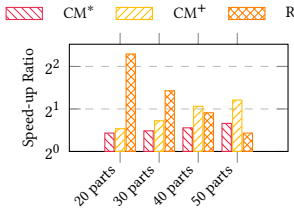


Figure 11: Scalability Study of Merge Efficiency on DEEP100M

Table 3: Graph Structure Analysis with Average Out-Degree (AD) and Out-Degree Percentiles $D_{5/50/99}$.

Method	Metric	SIFT	DEEP	GIST	IMAGE	ANTON	MARCO
Rebuild	AD	27.30	31.43	15.83	23.83	28.36	25.29
	$D_{5/50/99}$	11/26/50	12/30/59	2/11/58	7/21/58	10/26/59	9/22/59
FGIM	AD	20.48	22.60	12.05	15.24	19.08	14.55
	$D_{5/50/99}$	9/19/44	10/22/46	2/9/56	6/14/35	8/18/46	7/14/32
CM	AD	24.33	28.29	14.70	21.41	25.94	23.79
	$D_{5/50/99}$	10/23/49	11/27/59	2/11/58	7/18/57	9/24/58	8/21/58
RNSM	AD	23.56	28.48	19.46	23.94	25.59	25.14
	$D_{5/50/99}$	9/22/49	13/26/59	3/17/60	10/22/57	10/23/58	10/23/58

- **Merge Efficiency:** We quantify efficiency using the speed-up ratio relative to the CM, FGIM, *Rebuild* baselines, and the overlapping construction strategy employed by DiskANN.

Parameter Setting. We integrate our methods with various graph-based indexing algorithms, each with its own parameters. We leave the other parameters as default [62] and only consider the beam search effort ef. In particular, we examine several widely adopted graph-based AKNN search algorithms: HNSW [40], NSG [17], SSG [16], and τ-MNG [45]. We employ grid search to determine the optimal parameter values of k^+, k, and ef_L for Algorithm 2. For fair comparison, we set the same beam search parameter ef and maximum PG out-degree for CM and RNSM. The parameters of FGIM are set according to its default settings [66].

Implementation Environment. All experiments were conducted on a server running Ubuntu 22.04 LTS, equipped with dual Intel Xeon Platinum 8370C CPUs (2.80GHz, 32 physical cores each, supporting AVX-512) and 512 GB of DRAM. The proposed framework and baselines were implemented in C++ based on the `hnswlib` library [41] and compiled using GCC 13.1.0 with `-O3` optimization.

6.2 Experiment Results

Exp-1: Test of Indexes Merging Performance. We begin by evaluating the merging efficiency and subsequent search performance of our RNSM framework. For the setup, we randomly partition each dataset in Table 2 into two disjoint subsets of 0.5 million vectors and construct indexes using `hnswlib`. We compare RNSM against CM, FGIM, and a reconstruct-from-scratch baseline *Rebuild*.

Table. 4 reports the merge memory overhead (MO) and merge time (MT) across six datasets. FGIM has the highest MO, requiring 2.72×–4.07× the memory of RNSM because its KNNG transformation and refinement materialize a dense KNNG while simultaneously retaining the sub-indexes and the merged index, thereby duplicating vector data. We reduce this redundancy in both CM and RNSM by separating merged graph topology from the final searchable PG: input sub-indexes provide the vectors, the output stores only the new topology during merging, and vectors are attached after sub-indexes are released. CM has the lowest MO, while RNSM increases it by only 2.5%–11.1% for RKNN metadata. This is reasonable given that these metadata require only $O(nk)$ space ($k \leq 20$), substantially smaller than the $O(nD)$ space required by the dataset vectors. Merge on DEEP100M has a similar result: RNSM uses less MO than *Rebuild* and only 14.0% more than CM. For MT, RNSM delivers speed-up ratios up to 1.74× over CM, 3.86× over FGIM, and 9.92× over *Rebuild*. Notably, CM even outperforms the FGIM. According to Theorem 4.1 in FGIM [66], when the number of partitions is 2, the cross-index search in FGIM reduces to a naive search of CM. However, FGIM incurs additional overhead from refining KNN graphs and transforming the KNN graph into PG indexes.

Fig 9 further demonstrates the search performance (Recall vs. QPS/NDC). RNSM achieves a trade-off comparable to existing methods and the *Rebuild* baseline on all datasets. These results confirm that our framework substantially improves merge efficiency without compromising the search performance of the resulting index.

Table 4: Index statistics across datasets. MO denotes merge memory overhead (MB), and MT denotes merge time (s).

Method	Metric	SIFT	DEEP	GIST	IMAGE	ANTON	MARCO
Rebuild	MO	1530	1308	8386	6814	8911	8910
	MT	65.75	61.91	304.56	167.68	260.11	255.65
FGIM	MO	5361	5129	13037	11471	14467	14131
	MT	25.43	26.55	85.56	65.29	85.92	82.65
CM	MO	**1188**	**1139**	**4678**	**3892**	**4939**	**4939**
	MT	10.68	9.65	53.52	28.37	43.54	46.21
RNSM	MO	1317	1265	4797	4016	5066	5065
	MT	**7.52**	**6.98**	**42.31**	**16.90**	**29.96**	**26.51**

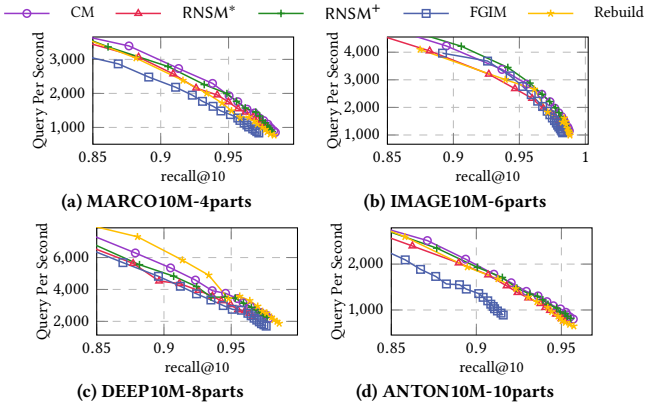


Figure 12: Recall@10-QPS Tradeoff for Large Scale Dataset

It is worth noting that the merged indexes can slightly outperform Rebuild in some configurations, a phenomenon also observed in FGIM [66]. This may arise because merging benefits from the accumulated construction effort of pre-built subgraphs and cross-partition searches. Moreover, graphs on sparse subsets yield longer edges, enabling beam search to cover more ground [68], while cross-partition edges preserve connectivity and recall.

Exp-2: Merged Graph Structure Analysis. We further analyse the graph index structure for merged indexes and rebuilt indexes. We investigate the graph degree distribution with the 5th, 50th, and 99th out-degree percentiles ($D_{5/50/99}$) and the average out-degree (AD). Table 3 shows that RNSM does not obtain its performance by constructing a denser graph. Although RNSM introduces cross-partition candidates, neighbor pruning keeps its AD comparable to CM and *Rebuild*. In contrast, FGIM yields the lowest AD and the smallest $D_{5/50/99}$ values. This sparsity reflects its KNNG transformation, where a small k is used for efficiency and leads to fewer edge candidates for edge pruning compared with *Rebuild*. Although CM and RNSM also use small edge candidate sets, they inherit edges that already survived pruning from a PG index, leading to similar out-degree distributions to *Rebuild*.

Exp-3: Test of Multiple Indexes Merging. We evaluate the search performance and merge efficiency when merging multiple indexes. We vary the number of partitions ($\{4, 6, 8, 10\}$) and assess robustness across different data distributions using both random (RNSM*) and cluster (RNSM+) partitioning. Experiments are conducted on DEEP10M, MARCO10M, IMAGE10M, and ANTON10M. Note that we compare RNSM* against both CM and FGIM, but compare RNSM+ only against CM, as FGIM is incompatible with cluster partitions. Fig 10 reports the merge efficiency on MARCO10M (full results in Appendix). Our methods achieve substantial speedups over baselines. For example, on MARCO10M, RNSM* delivers speed-up ratios of up to 2.31× over CM, 1.38× over FGIM, and 4.26× over *Rebuild*, while RNSM+ achieves a 1.90× speed-up over CM. Regarding search quality, as shown in Fig 12, our framework maintains performance comparable to baselines across varying partition numbers and distributions.

Table 5: Comparison of Overlap and RNSM*. (Time: seconds, QPS_x: Query Per Second with recall@10=x)

Max Size	Dataset	Overlap DiskANN			RNSM*		
		Time	$QPS_{0.90}$	$QPS_{0.95}$	Time	$QPS_{0.90}$	$QPS_{0.95}$
5M	DEEP10M	1247	4538	3042	**630**	**4554**	**3175**
	ANTON10M	4116	**1672**	783	**2928**	1637	**819**
	MARCO10M	4410	2021	1436	**3138**	**2573**	**1751**
	IMAGE10M	2575	**3509**	2314	**1518**	3423	**2395**
2.5M	DEEP10M	1206	**5295**	3225	**767**	4901	**3493**
	ANTON10M	4084	**1803**	780	**2649**	1791	**797**
	MARCO10M	4446	2308	1528	**3295**	**2429**	**1662**
	IMAGE10M	3185	3603	**2550**	**1582**	**3807**	2456

We further assess the scalability of our methods regarding dataset size and partition count on the large-scale DEEP100M dataset. We partition the data into $\{20, 30, 40, 50\}$ subsets using both random and cluster partitioning and merge them using RNSM* and RNSM+, respectively. We exclude FGIM here as its Minimum Candidate Set Strategy requires ef to grow with partition number ($ef > m - 1$). Experimentally, its speedup over *Rebuild* drops from 1.97× with 4 partitions to 0.99× with 10 on IMAGE10M (see Appendix Exp-c). Results in Fig 11 show that RNSM consistently speeds up merging compared with the CM and Rebuild baseline with comparable search performance (see Appendix Exp-d). Showing that RNSM is suited for massive multi-index merging scenarios.

Exp-4: Comparison with Overlap-based Method. We compare our framework against the overlapping partition strategy used in DiskANN on 10M-scale datasets. Unlike DiskANN, which partitions data into overlapping shards (exactly one duplication for each vector) and builds sub-indexes from scratch, RNSM directly merges existing sub-indexes. For a fair comparison, we align the maximum shard size of Overlap with the total vector count of the indexes merged by RNSM (*e.g.*, if Overlap uses a 5M shard size, RNSM merges four 2.5M sub-indexes). We also use the same index of Overlap to focus on the merge algorithm itself. We include both partitioning and construction time for both methods and report the QPS with 0.95 recall in Table 5. Our method achieves comparable search efficiency while significantly reducing index construction time. Specifically, RNSM is consistently faster across all datasets even without any pre-built indexes, delivering speedups of up to 2.01× compared to the scratch-build approach of DiskANN. Furthermore, the search performance is comparable to the Overlap method, with higher $QPS_{0.95}$ in 7 of 8 configurations.

Exp-5: Test of Incremental Merging. We evaluate incremental merging on 10M-scale datasets by splitting each dataset into ten 1M-vector partitions. Starting from one partition, we merge one additional 1M-vector sub-index into the current merged index at each step until all ten partitions form the final 10M-vector index. As

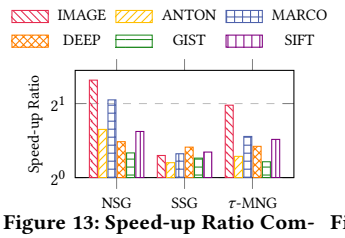


Figure 13: Speed-up Ratio Comparison of Different Methods

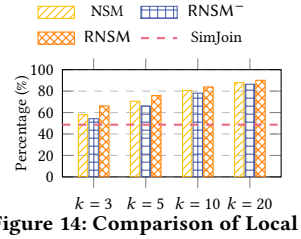
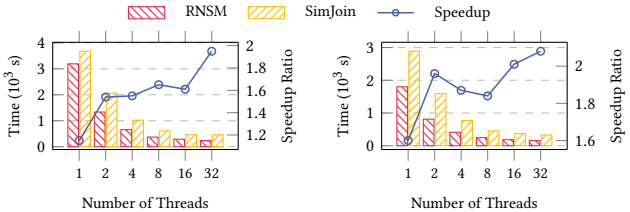
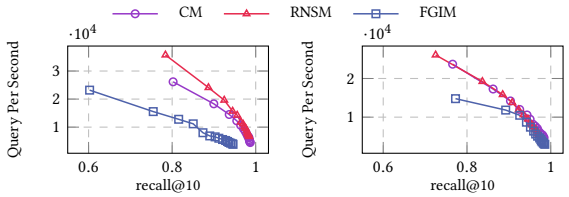


Figure 14: Comparison of Local Sliding Ratio with SIFT1M



(a) MARCO10M **(b) IMAGE10M**

Figure 15: Parallel Scalability Comparison



(a) IMAGE10M **(b) MARCO10M**

Figure 16: Recall@10-QPS Tradeoff for Incremental Merge

shown in Fig 16, RNSM achieves search performance comparable to or better than the baselines. In terms of merge efficiency, RNSM delivers speedups of up to 1.57× over CM and 6.44× over FGIM.

Exp-6: Applicability Study. We evaluate RNSM on three additional PG indexes: NSG, SSG, and τ-MNG. We use recommended settings for NSG and SSG [62] and implement τ-MNG on the NSG codebase [45]. Fig 13 reports RNSM achieves speedups of up to 2.49×, 1.33×, and 1.97× for NSG, SSG, and τ-MNG, respectively. RNSM also maintains search performance comparable to CM across these indexes (see Appendix Exp-b), demonstrating its broad applicability.

Exp-7: Effects of Reverse Neighbor Pivots Selection. We analyze the impact of RKNN-based pivot selection on the fraction of Local Sliding operations. On SIFT1M with $k \in \{3, 5, 10, 20\}$, we compare four variants: (1) NSM: randomly select pivots, slide to KNNs; (2) SimJoin: MST roots as pivots, slide to internal nodes and leaves; (3) RNSM$^-$: randomly select pivots, slide to RKNNs; and (4) RNSM: pivots and corresponding followers selected by our greedy algorithm. As shown in Fig 14, SimJoin has the lowest ratio, and its fixed MST topology determines the sliding proportion. For the other methods, the ratio increases with k. RNSM rises from 66.2% at $k = 3$ to 90.1% at $k = 20$, showing that larger k exposes more followers. RNSM consistently outperforms all other methods. RNSM$^-$ underperforms NSM because NSM iteratively propagates previous results instead of sliding to a fixed follower set.

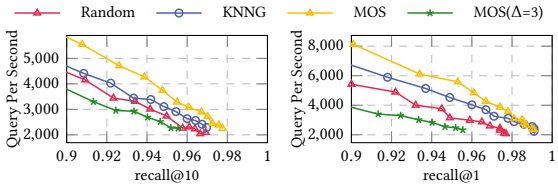


Figure 17: Merge Order Selection Comparison on DEEP10M

Exp-8: Parallel Scalability. We compare RNSM and SimJoin on MARCO10M and IMAGE10M using {1, 2, 4, 8, 16, 32} threads at comparable merged-index search quality for parallelism comparison. Fig 15 shows that RNSM outperforms SimJoin across all thread configurations, with its advantage increasing with more threads: from 1.15× to 1.95× on MARCO10M and from 1.60× to 2.08× on IMAGE10M. The MST propagation of SimJoin introduces inter-node dependencies that limit parallelism, while RNSM decouples sliding through independent pivot selection.

Exp-9: Effects of MOS Technique. We validate the effectiveness of the MOS strategy for merging multiple partitions. On DEEP10M split into 10 partitions, we compare MOS with a random regular graph, a KNNG with the same average degree, and an MOS variant with diameter upper bound $\Delta = 3$. We exclude MST, Star, and Path because they fail to reach 0.9 recall in Fig 4. Fig 17 shows that MOS consistently yields the best search performance.

7 Related Works

HNSW-merger. HNSW-merger [33] proposed merging HNSW indexes via cross-querying. Specifically, they use one-way cross-querying from the smaller to the larger index with a small candidate set ef. It further exploits the HNSW hierarchy to cache higher-layer entry points. However, they handle multi-index merges solely by incrementally merging the two largest indexes, limiting speedups.

Similarity Join. SimJoin [67] was originally proposed to report all cross-partition vector pairs within a distance threshold. Cross-index KNN discovery of index merging is a top-k variant of this task. However, it requires an MST over one partition and cannot directly merge two PG indexes.

KNNG Merge. S-merge [78] connects two KNNGs with random edges and refines them using NNDecent [14]. Multi-way Merge [75] merges multiple KNNGs in one pass by cross-matching only newly added cross-index candidates. Batch processing in NNDecent lets KNNGs converge faster than PG-based methods [74], but KNNGs are generally less effective for AKNN search than navigational graph indexes [5, 37, 62, 63].

Overlapping Partitioning. This strategy is often employed for billion-scale graph index construction under memory constraints (*e.g.*, DiskANN). The dataset is partitioned into overlapping clusters via k-means, where each data point is assigned to at least 2 nearest partitions. Sub-indexes are constructed for each partition, and a compact graph is subsequently merged by concatenating the neighborhoods of identical nodes across different subgraphs. However, it requires clustering the dataset and building sub-indexes from scratch, making it inapplicable to merging existing indexes.

8 Conclusion and Future Work

In this paper, we addressed the challenge of merging multiple proximity graph indexes to enable efficient multi-index searching. By identifying the core components of graph merging, we proposed the Reverse Neighbor Sliding Merge (RNSM) algorithm to facilitate pairwise merging. Furthermore, based on merge order analysis, we designed the Merge Order Selection (MOS) algorithm. MOS is suitable for both random and cluster partitions, and it reduces redundancy in pairwise processing by pruning unnecessary merges. Our experimental results demonstrate that our framework significantly improves merging efficiency while preserving the comparable search performance characteristic of graph indexes. Notably, our method exhibits robust scalability, performing effectively even with large-scale datasets.

In practice, systems may involve overlapping, unequal-sized, or mixed partitions rather than simplified disjoint random or cluster partitions. While our method can be directly extended to these settings, it would be suboptimal, and designing more sophisticated merge-cost models tailored to them remains future work. Moreover, the three proxies used in MOS are indirect and empirical. Richer size-aware, geometric, or learned proxies that directly optimize merge time and quality remain future work.

References

[1] 2026. The Faiss Library. *IEEE* (2026).

[2] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proceedings of the VLDB Endowment* (2023), 3433–3446.

[3] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache Locality Is Not Enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment* (2015), 288–299.

[4] Sunil Arya and David M. Mount. 1993. Approximate Nearest Neighbor Queries in Fixed Dimensions. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '93)*. Society for Industrial and Applied Mathematics, USA, 271–280.

[5] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* (2020), 101374.

[6] Artem Babenko and Victor Lempitsky. 2015. The Inverted Multi-Index. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2015), 1247–1260.

[7] Jonathan Bac, Evgeny M Mirkes, Alexander N Gorban, Ivan Tyukin, and Andrei Zinovyev. 2021. Scikit-dimension: a python package for intrinsic dimension estimation. *Entropy* 23, 10 (2021), 1368.

[8] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover Trees for Nearest Neighbor. In *Proceedings of the 23rd International Conference on Machine Learning - ICML '06*. ACM Press, Pittsburgh, Pennsylvania, 97–104.

[9] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023*. ACM, Austin TX USA, 3225–3235.

[10] T. Cover and P. Hart. 1967. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27. doi:10.1109/TIT.1967.1053964

[11] Abhinandan S. Das, Mayur Datar, Ashutosh Garg, and Shyam Rajaram. 2007. Google News Personalization: Scalable Online Collaborative Filtering. In *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*. Association for Computing Machinery, New York, NY, USA, 271–280. doi:10.1145/1242572.1242610

[12] Sanjoy Dasgupta and Yoav Freund. 2008. Random Projection Trees and Low Dimensional Manifolds. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*. ACM, Victoria British Columbia Canada, 537–546. doi:10.1145/1374376.1374452

[13] Mayur Datar, Piotr Indyk, Nicole Immorlica, and Vahab S Mirrokni. 2004. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions. *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms* (2004).

[14] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web*. ACM, Hyderabad India, 577–586.

[15] Karima Echihabi, Panagiota Fatourou, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2022. Hercules against Data Series Similarity Search. *Proc. VLDB Endow.* (2022), 2005–2018.

[16] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022), 4139–4150.

[17] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* (2019), 461–474.

[18] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-Sensitive Hashing Scheme Based on Dynamic Collision Counting. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. ACM, Scottsdale Arizona USA, 541–552.

[19] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* (2025), 202:1–202:26.

[20] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: With Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* (2023), 1–27.

[21] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (May 2024), 27 pages. doi:10.1145/3654970

[22] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. doi:10.1109/TPAMI.2013.240

[23] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. (1999), 518–529.

[24] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* (2025), 80:1–80:26.

[25] Magnús M. Halldórsson. 1993. Approximating the Minimum Maximal Independence Number. *Inform. Process. Lett.* (1993), 169–172.

[26] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Las Vegas, NV, USA, 5713–5722.

[27] A. J. Hoffman and R. R. Singleton. 1960. On Moore graphs with diameters 2 and 3. *IBM J. Res. Dev.* 4, 5 (Nov. 1960), 497–504. doi:10.1147/rd.45.0497

[28] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing - STOC '98*. ACM Press, Dallas, Texas, United States, 604–613.

[29] Piotr Indyk and Haike Xu. 2023. Worst-Case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations. *Advances in Neural Information Processing Systems* (2023), 66239–66256.

[30] Robert W. Irving. 1991. On approximating the minimum independent dominating set. *Inf. Process. Lett.* 37, 4 (1991), 197–200. doi:10.1016/0020-0190(91)90188-N

[31] H Jégou, M Douze, and C Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2011), 117–128.

[32] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3, Article 148 (June 2025), 26 pages. doi:10.1145/3725399

[33] Chenzhe Jin, Yunan Zhang, Jiayi Liu, and Jianguo Wang. 2026. Efficient Vector Index Merging in Vector Databases. 4, 1 (2026). https://doi.org/10.1145/3786645

[34] Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.

[35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.

[36] Binhong Li, Xiao Yan, and Shangqi Lu. 2025. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* (2025).

[37] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. 1475–1488 pages.

[38] Ying Liu, Dengsheng Zhang, Guojun Lu, and Wei-Ying Ma. 2007. A Survey of Content-Based Image Retrieval with High-Level Semantics. *Pattern Recognition* 40, 1 (2007), 262–282. doi:10.1016/j.patcog.2006.04.045

[39] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs. *Information Systems* (2014), 61–68.

[40] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.

IEEE Transactions on Pattern Analysis and Machine Intelligence (2020), 824–836.

[41] Yu A Malkov and Dmitry A Yashunin. 2020. hnswlib. https://github.com/nmslib/hnswlib.

[42] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '24)*. Association for Computing Machinery, New York, NY, USA, 270–285.

[43] Gonzalo Navarro. 2002. Searching in Metric Spaces by Spatial Approximation. *The VLDB Journal The International Journal on Very Large Data Bases* (2002), 28–46.

[44] Zhaojie Niu, Xinhui Tian, Xindong Peng, and Xing Chen. 2025. BlendHouse: A Cloud-Native Vector Database System in ByteHouse. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 4332–4345.

[45] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1, Article 54 (2023), 27 pages. doi:10.1145/3588908

[46] James Philbin, Ondrej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. 2007. Object retrieval with large vocabularies and fast spatial matching. In *2007 IEEE Conference on Computer Vision and Pattern Recognition*. 1–8. doi:10.1109/CVPR.2007.383172

[47] Runwen Qiu and Jing Tang. 2025. Efficient Approximate Nearest Neighbor Search via Hemi-Sphere Centroids Graph. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[48] J. Ben Schafer, Dan Frankowski, Jon Herlocker, and Shilad Sen. 2007. Collaborative Filtering Recommender Systems. In *The Adaptive Web: Methods and Strategies of Web Personalization*, Peter Brusilovsky, Alfred Kobsa, and Wolfgang Nejdl (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 291–324. doi:10.1007/978-3-540-72079-9_9

[49] Zhenpei Shan, Defu Zhang, and Yunqi Lei. 2025. DF-LSH: An Efficient Double Filters Locality Sensitive Hashing for Approximate Nearest Neighbor Search. *Engineering Applications of Artificial Intelligence* (2025), 110742.

[50] Yang Shi, Yiping Sun, Jiaolong Du, Xiaocheng Zhong, Zhiyong Wang, and Yao Hu. 2025. Scalable Overload-Aware Graph-Based Index Construction for 10-Billion-Scale Vector Similarity Search. In *Companion Proceedings of the ACM on Web Conference 2025 (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1303–1307.

[51] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613* (2021).

[52] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–29.

[53] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 13766–13776.

[54] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: Solving c-Approximate Nearest Neighbor Queries in High Dimensional Euclidean Space with a Tiny Index. *Proceedings of the VLDB Endowment* (2014), 1–12.

[55] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2010. Efficient and Accurate Nearest Neighbor and Closest Pair Search in High-Dimensional Space. *ACM Transactions on Database Systems* (2010), 1–46.

[56] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2024. DB-LSH 2.0: Locality-Sensitive Hashing With Query-Based Dynamic Bucketing. *IEEE Transactions on Knowledge and Data Engineering* (2024), 1000–1015.

[57] Godfried T. Toussaint. 1980. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12, 4 (Jan. 1980), 261–268. doi:10.1016/0031-3203(80)90066-7

[58] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 188 (June 2025), 27 pages. doi:10.1145/3725325

[59] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627.

[60] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627. doi:10.1145/3448016.3457550

[61] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proceedings of the ACM on Management of Data (SIGMOD)* (2025).

[62] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2021), 1964–1978.

[63] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47 (2023), 3–21. https://api.semanticscholar.org/CorpusID:265509541

[64] Jiuqi Wei, Xiaodong Lee, Zhenyu Liao, Themis Palpanas, and Botao Peng. 2025. Subspace Collision: An Efficient and Accurate Framework for High-dimensional Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* (2025), 79:1–79:29.

[65] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2024), 2241–2254.

[66] Zekai Wu, Jiabao Jin, Peng Cheng, Xiaoyao Zhong, Lei Chen, Yongxin Tong, Zhitao Shen, Jingkuan Song, Heng Tao Shen, and Xuemin Lin. 2026. FGIM: a Fast Graph-based Indexes Merging Framework for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 4 (2026), 1–27.

[67] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Fast Approximate Similarity Join in Vector Databases. *Proceedings of the ACM on Management of Data* (2025), 1–26.

[68] Ming Yang, Yuzheng Cai, and Weiguo Zheng. 2024. CSPG: Crossing Sparse Proximity Graphs for Approximate Nearest Neighbor Search. *Advances in Neural Information Processing Systems* (2024), 103076–103100.

[69] Mingyu Yang, Liuchang Jing, Wentao Li, and Wei Wang. 2024. Quantization Meets Projection: A Happy Marriage for Approximate k-Nearest Neighbor Search. *arXiv preprint arXiv:2411.06158* (2024).

[70] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2025. Effective and General Distance Computation for Approximate Nearest Neighbor Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1098–1110.

[71] Mingyu Yang, Wentao Li, Zhitao Shen, Chuan Xiao, and Wei Wang. 2025. ESG: Elastic Graphs for Range-Filtering Approximate k-Nearest Neighbor Search. *arXiv preprint arXiv:2504.04018* (2025).

[72] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2025), 1825–1838.

[73] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. *arXiv preprint arXiv:2510.17326* (2025).

[74] Cheng Zhang, Jianzhi Wang, Wan-Lei Zhao, and Shihai Xiao. 2025. Highly Efficient Disk-based Nearest Neighbor Search on Extended Neighborhood Graph. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Padua Italy, 2513–2523.

[75] Cheng Zhang, Wan-Lei Zhao, Shihai Xiao, Jiajie Yao, and Xuecang Zhang. 2025. Towards the Distributed Large-scale k-NN Graph Construction by Graph Merge. *arXiv preprint arXiv:2509.11697* (2025).

[76] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.

[77] Kang Zhao, Pan Pan, Yun Zheng, Yanhao Zhang, Changxu Wang, Yingya Zhang, Yinghui Xu, and Rong Jin. 2019. Large-Scale Visual Search with Binary Distributed Graph at Alibaba. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 2567–2575.

[78] Wan-Lei Zhao, Hui Wang, Peng-Cheng Lin, and Chong-Wah Ngo. 2022. On the Merge of k-NN Graph. *IEEE Transactions on Big Data* 8, 6 (2022), 1496–1510. doi:10.1109/TBDATA.2021.3101517

[79] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proceedings of the VLDB Endowment* (2023), 1979–1991.

[80] Xiaoyao Zhong, Jiabao Jin, Peng Cheng, Mingyu Yang, Haoyang Li, Zhitao Shen, Heng Tao Shen, and Jingkuan Song. 2025. EnhanceGraph: A Continuously Enhanced Graph-based Index for High-dimensional Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2506.13144* (2025).

[81] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2025), 5017–5030.

Appendix

Proof

Proof of Theorem 4.2

Proof. To prove that the Minimum Cost Independent Dominating Pivot Selection (IDPS) problem is NP-hard, we establish a polynomial-time reduction from the NP-complete **Minimum Independent Dominating Set (MIDS)** problem [30].

First, we define the decision versions of both problems.

- **MIDS-Decision:** Given an undirected graph $G' = (V', E')$ and a positive integer k, does there exist an independent dominating set $D \subseteq V'$ with $|D| \leq k$? (i.e., every $v \notin D$ has a neighbor in D, and no two nodes in D are adjacent.)
- **IDPS-Decision:** Given a vector dataset X, a graph $G = (V, E)$, a distance function $\delta(\cdot, \cdot)$, a naive search cost γ, and a target cost C, does there exist a pivot subset $P \subseteq V$ satisfying the independent dominating set constraint such that $\mathcal{L}(P) \leq C$?

NP Membership. IDPS-Decision is in NP: given a candidate P, one can verify in polynomial time that (i) every $v \notin P$ has a neighbor in P (domination), (ii) no two nodes in P are adjacent (independence), and (iii) $\mathcal{L}(P) \leq C$.

Polynomial-Time Construction. Given an instance $\langle G', k \rangle$ of MIDS-Decision, we construct an instance $\langle X, G, \delta, \gamma, C \rangle$ of IDPS-Decision as follows:

(1) Let $G = G'$, so $V = V'$ and $E = E'$.
(2) Map each node $v_i \in V$ to a distinct vector $x_i \in X$. Define the discrete metric: $\delta(x_u, x_u) = 0$ and $\delta(x_u, x_v) = 1$ for all $u \neq v$. (This trivially satisfies all metric space properties.)
(3) Set $\gamma = 2$, $\beta = 1$, and $C = k + |V|$.

This construction runs in polynomial time $O(|V|^2)$.

Cost Simplification. For any P satisfying the independent dominating set constraint, P is in particular a dominating set, so every $v \notin P$ has a neighbor $p \in P \cap N(v, G)$. With $\delta \equiv 1$ for distinct nodes, the sliding cost is:

$$\mathcal{L}_{slide}(P) = \sum_{v \in V \setminus P} \min_{p \in P \cap N(v,G)} \delta(x_v, x_p) = |V| - |P|$$

Substituting $\gamma = 2$:

$$\mathcal{L}(P) = 2|P| + (|V| - |P|) = |P| + |V|$$

Correctness of Reduction.

- **(⇒)** Suppose there exists an independent dominating set D in G' with $|D| \leq k$. Let $P = D$. Since $G = G'$, P is an independent dominating set in G, satisfying both IDPS constraints. The total cost is $\mathcal{L}(P) = |P| + |V| \leq k + |V| = C$. Thus, a valid IDPS solution exists.
- **(⇐)** Suppose there exists a pivot subset $P \subseteq V$ satisfying the IDPS constraints with $\mathcal{L}(P) \leq C$. By the IDPS constraints, P is an independent dominating set of $G = G'$. From $\mathcal{L}(P) \leq C$ we get $|P| + |V| \leq k + |V|$, so $|P| \leq k$. Thus, an independent dominating set of size at most k exists in G'.

Since MIDS-Decision is NP-complete and MIDS-Decision $\leq_p$ IDPS-Decision, IDPS-Decision is NP-complete. Consequently, the IDPS optimization problem is NP-hard. □

Extra Experiments

Exp-a: Runtime Breakdown of RNSM. In this section, we analyze the execution time of RNSM by decomposing it into its core components (32 threads): neighbor expansion, reverse inversion and pivot selection, and execution of merging. As shown in Fig 18, the merging step dominates runtime across all datasets, accounting for more than 83.1% of total time, since it involves the main graph-update operations driven by the sliding cost model. Neighbor expansion contributes 8.7%–15.3%, reflecting the overhead of retrieving nearest neighbors of every node from the source index. Pivot selection is consistently negligible (≤5.0%), confirming that the sequential pivot in Algorithm 2 adds virtually no overhead.

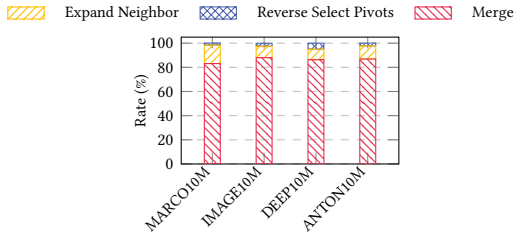


Figure 18: Time Breakdown of RNSM Components

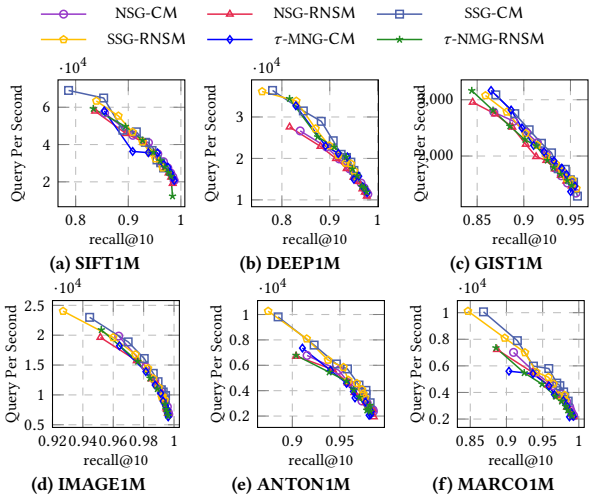
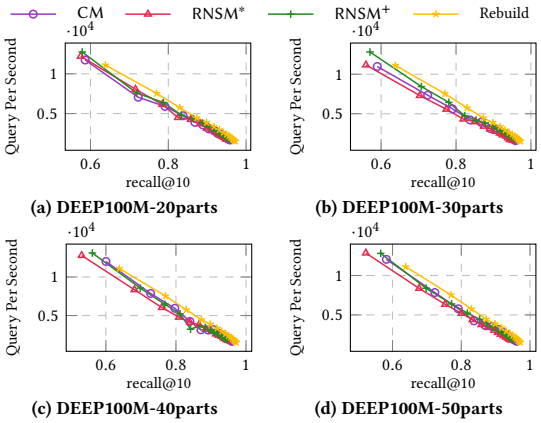


Figure 19: Recall@10-QPS Tradeoff of Different Methods

Exp-b: Applicability Study of Search Performance. This section evaluates the applicability of RNSM across different graph index structures beyond HNSW. We compare RNSM against the Cross-query Merge (CM) baseline on three representative proximity graph types: NSG [17], SSG [16], and τ-MNG [45].

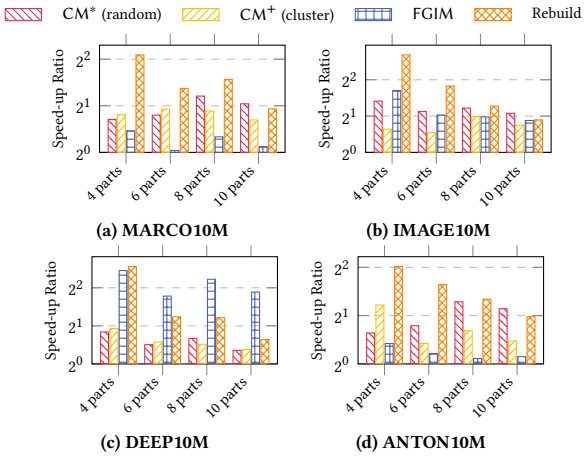
Fig 19 presents the complete results. Across all configurations, RNSM consistently maintains search performance comparable to or better than CM. This demonstrates that RNSM preserves the navigability and search efficiency of the underlying graph indexes, regardless of their specific construction algorithms or edge selection policies.



(a) DEEP100M-20parts (b) DEEP100M-30parts

(c) DEEP100M-40parts (d) DEEP100M-50parts

Figure 20: Recall@10-QPS Tradeoff for More Data Partitions



(a) MARCO10M (b) IMAGE10M

(c) DEEP10M (d) ANTON10M

Figure 21: Speed-up Ratio Comparison across Different Partitions

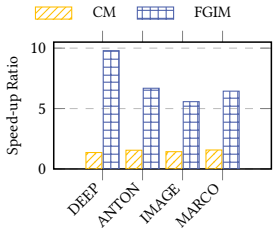


Figure 22: Incremental Merge Speed-up Ratio on 10M Datasets.

Exp-c: Comprehensive Results for Exp-3. This section presents the comprehensive experimental results for Exp-3 (Multiple Index Merging). It covers all evaluated datasets (DEEP10M, MARCO10M, IMAGE10M, and ANTON10M) using both random and cluster partitioning strategies across varying partition counts $\{4, 6, 8, 10\}$.

Fig 21 reports the complete speedup ratios across all datasets and partition numbers. Our method shows the best merge efficiency in all cases, achieving up to 2.33×, 5.48×, and 6.43× acceleration compared to CM, FGIM, and Rebuild, respectively.

Fig 23 presents the complete recall@10-QPS tradeoff curves for all experimental configurations. The results demonstrate that our RNSM and MOS framework maintains the expected search performance of graph indexes and outperforms the existing merge method, FGIM.

Exp-d: Search Performance of Scalability Study for Exp-3. This experiment complements the DEEP100M scalability study in Exp-3: Test of Multiple Indexes Merging. We partition DEEP100M into $\{20, 30, 40, 50\}$ sub-indexes and evaluate RNSM* and RNSM+ on random and cluster partitions, using CM as the baseline. Fig 20 reports the recall@10-QPS tradeoff curves for the scalability experiments on DEEP100M and the merge of {20,30,40,50} indexes. Results show that our RNSM+MOS method achieves comparable search performance to the CM and Rebuild baselines across all settings.

Exp-e: Incremental Merge Efficiency for Exp-5. This experiment complements the search-performance results of Exp-5 with the corresponding merge-efficiency measurements. For each dataset, we sum the merge time over the nine incremental steps from 1M to 10M vectors and report the cumulative merge time of each baseline divided by that of RNSM; a ratio greater than one therefore indicates a speedup by RNSM.

As shown in Fig 22, RNSM consistently reduces the cumulative merge time across all four datasets. Its speedup ranges from 1.36× to 1.57× over CM and from 5.57× to 9.79× over FGIM. On MARCO10M, RNSM achieves the 1.57× and 6.44× speedups over CM and FGIM, respectively, reported in Exp-5. Together with the recall@10–QPS results in Fig 16, these measurements show that RNSM improves incremental merge efficiency while maintaining comparable search performance.

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026

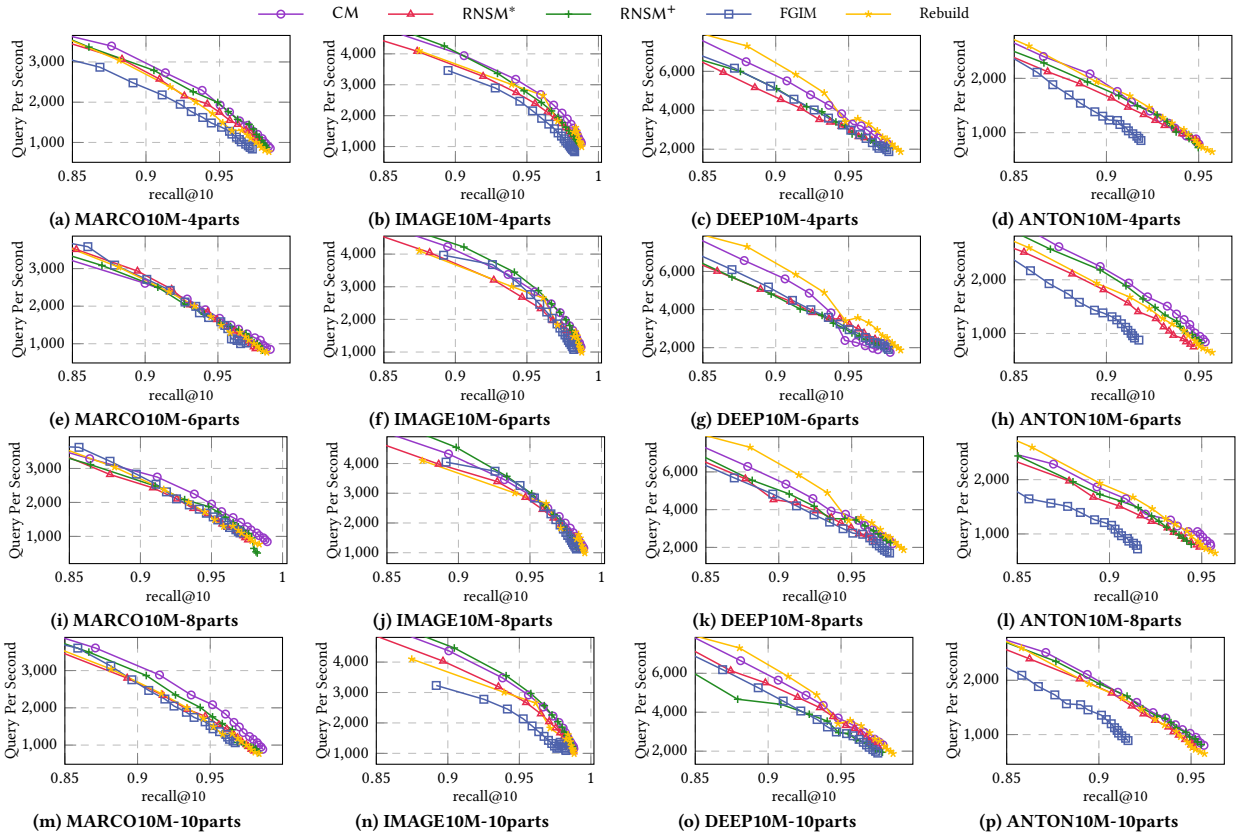


Figure 23: Recall@10-Query Per Second Tradeoff for Merge Methods